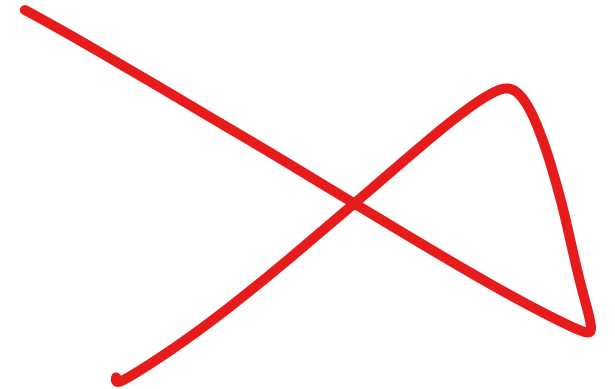
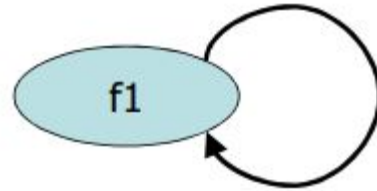


# Recursion and Tower of Hanoi



# Recursion

- The process in which a function calls itself is called recursion, and the corresponding function is called as recursive function.



- Why Recursion?
  - Recursion is a very useful and powerful technique.
  - Using recursive algorithm, certain problems can be solved quite easily. Examples of such problems are:
    - Some algorithms that we have already covered (Merge Sort, Binary Search, etc.)
    - Towers of Hanoi (TOH)
    - Inorder/Preorder/Postorder Tree Traversals
    - DFS of Graph, etc.

# Recursion

## How a problem is solved using recursion?

- In the recursive program,
  - the solution to the base case is provided [**basis step**], and
  - the solution of the bigger problem is expressed in terms of smaller problems [**recursive step**].

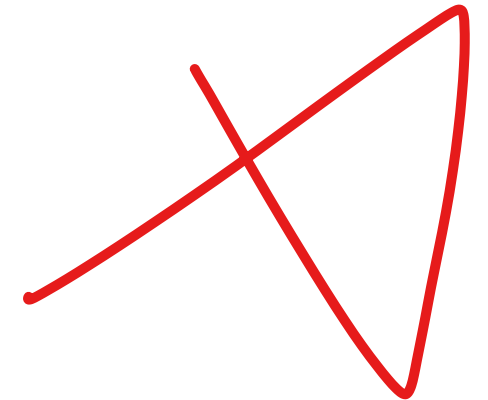
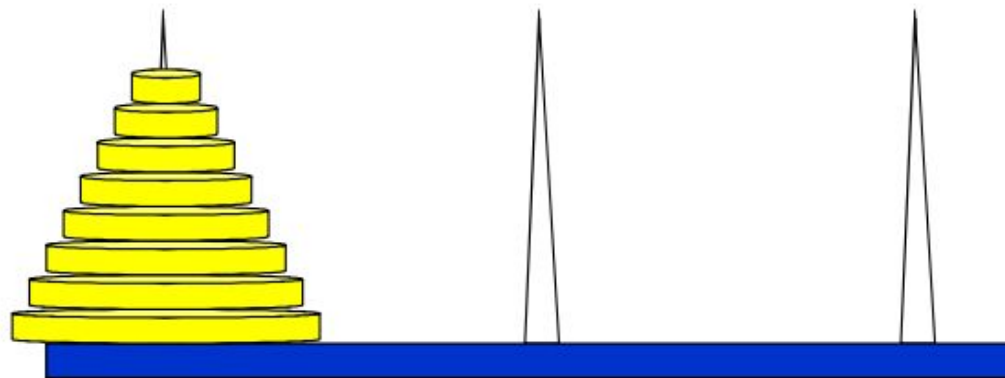
```
if this is a simple case
    solve it
else
    redefine the problem using recursion
```

## Why Stack Overflow error occurs in recursion?

- If the base case is not reached or not defined, then the stack overflow problem may arise.

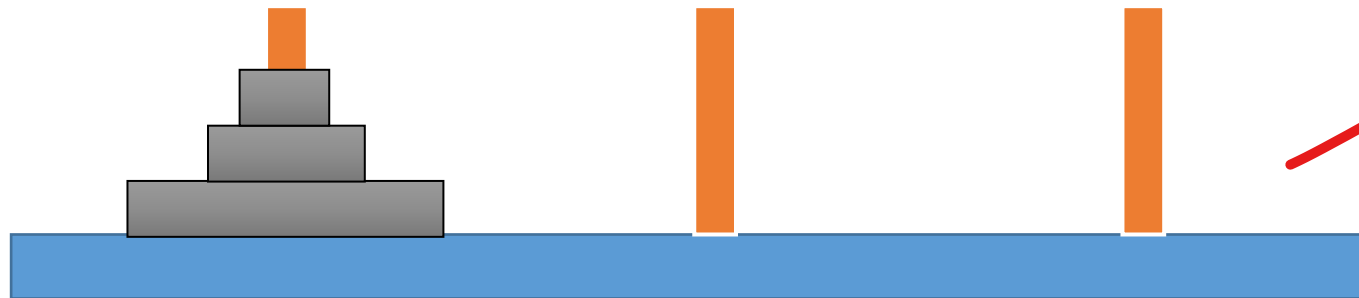
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
  - Assume there are **three towers available**
  - The **constraint** is that the larger disk can never be placed on top of a smaller disk.
  - **Only one disk** can be moved at each time



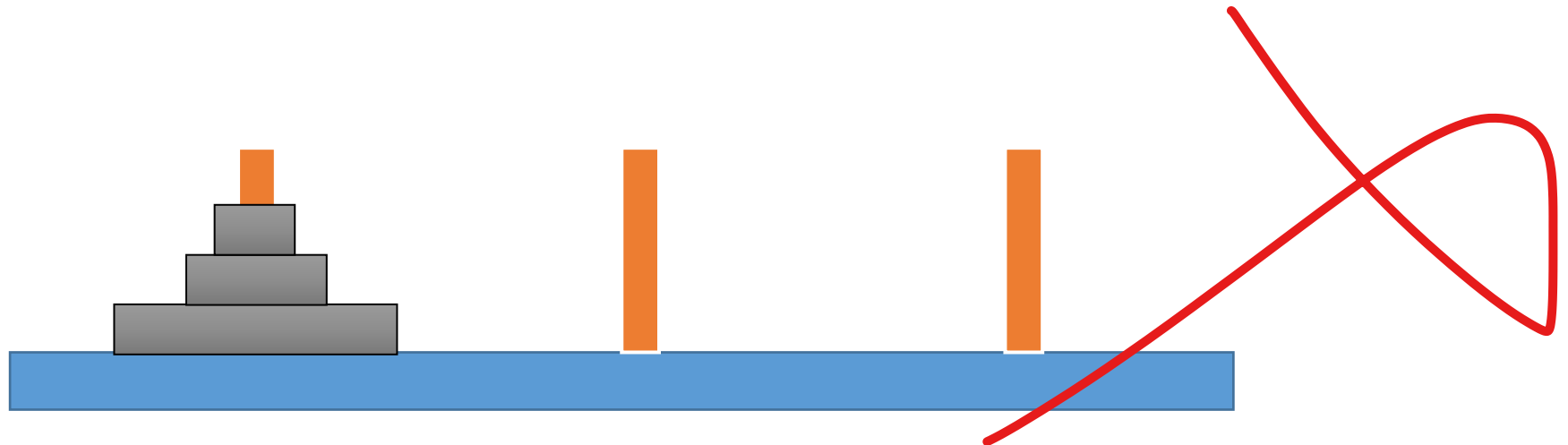
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
- ➡
- Assume there are **three towers available**
  - The **constraint** is that the larger disk can never be placed on top of a smaller disk.
  - **Only one disk** can be moved at each time



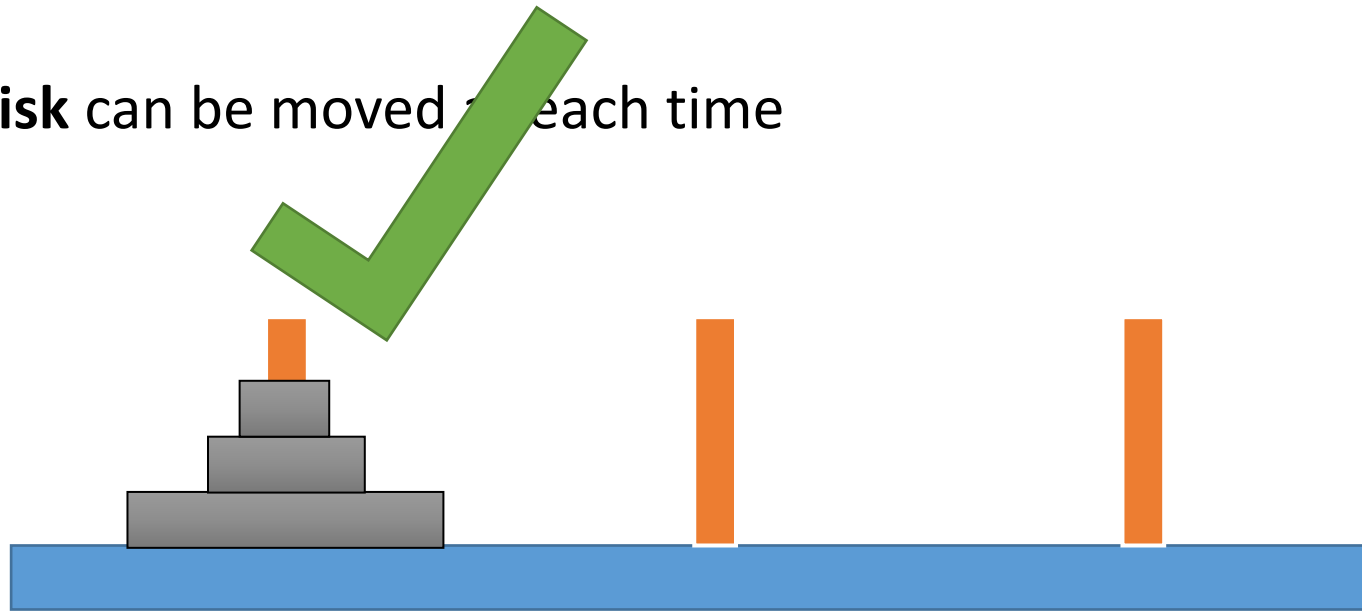
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
  - Assume there are **three towers available**
- ➔ • The **constraint** is that the larger disk can never be placed on top of a smaller disk.
- **Only one disk** can be moved at each time



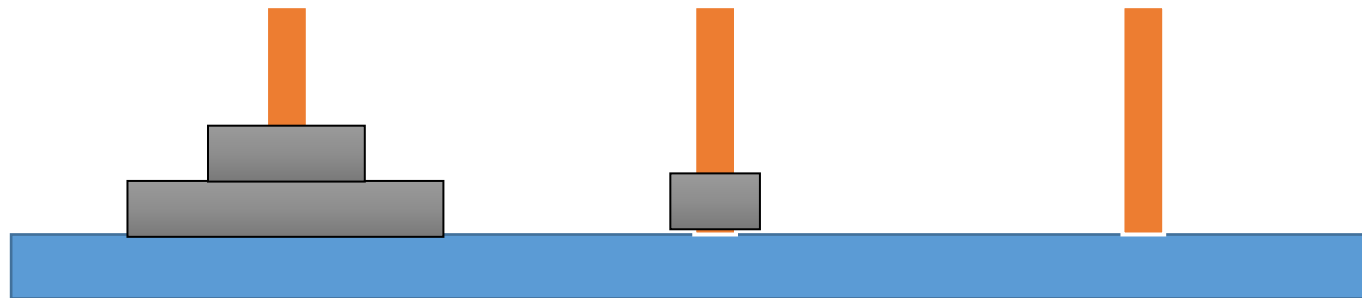
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
  - Assume there are **three towers available**
  - ➔ • The **constraint** is that the larger disk can never be placed on top of a smaller disk.
  - **Only one disk** can be moved at each time



# Tower of Hanoi Problem

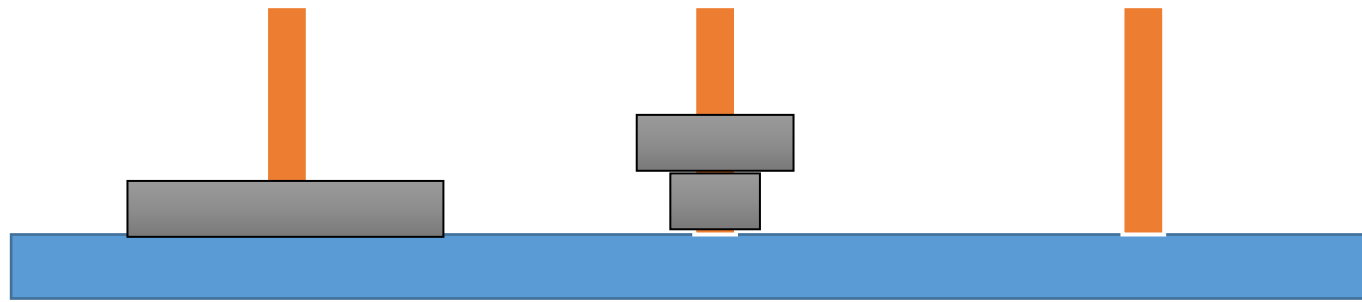
- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
  - Assume there are **three towers available**
  - ➔ • The **constraint** is that the larger disk can never be placed on top of a smaller disk.
  - **Only one disk** can be moved at each time





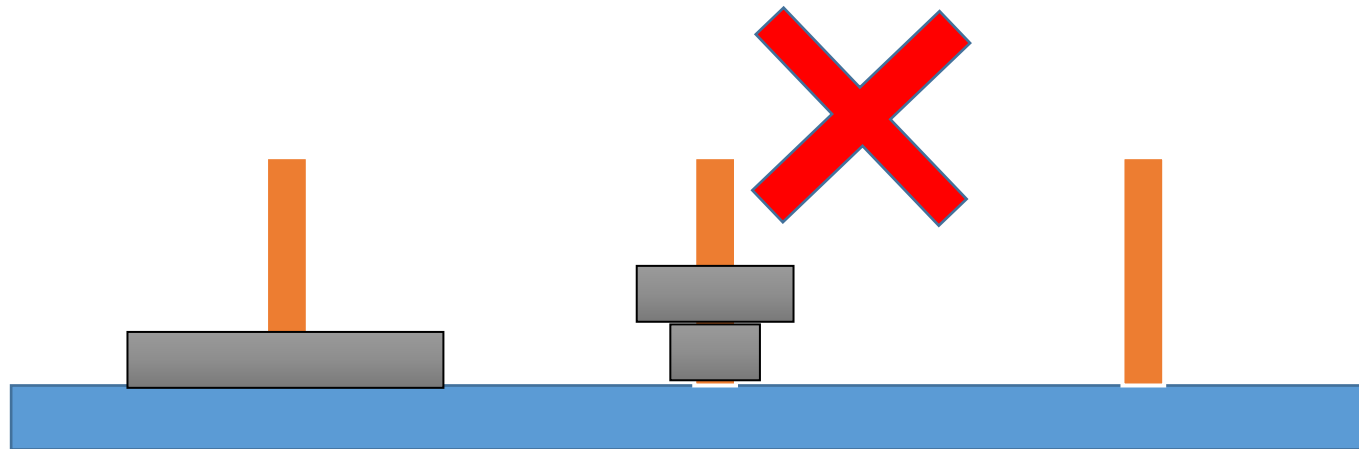
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
  - Assume there are **three towers available**
- ➔ • The **constraint** is that the larger disk can never be placed on top of a smaller disk.
- **Only one disk** can be moved at each time



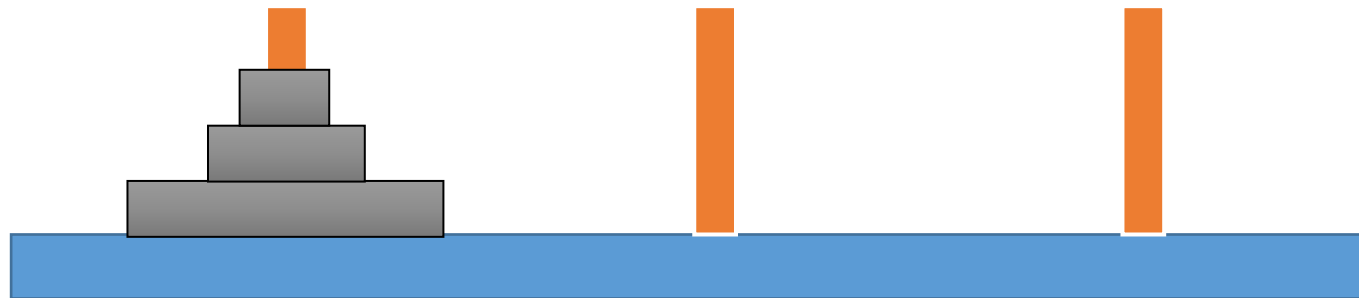
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
  - Assume there are **three towers available**
- ➔ • The **constraint** is that the larger disk can never be placed on top of a smaller disk.
- **Only one disk** can be moved at each time



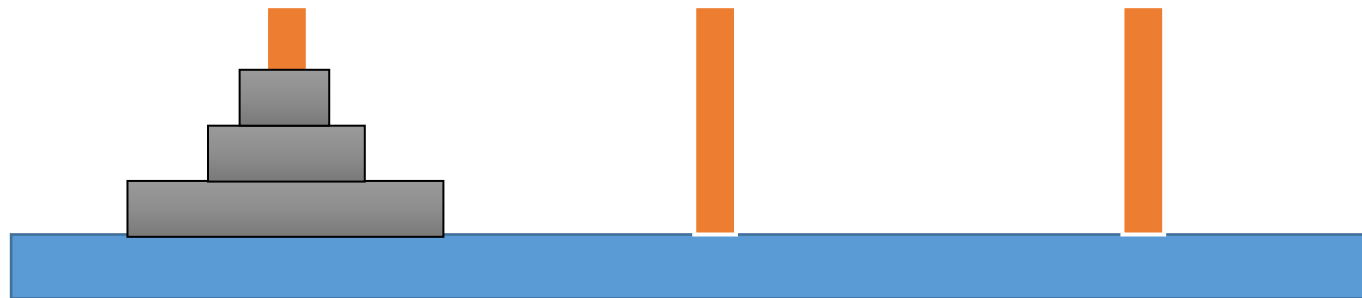
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
    - Assume there are **three towers available**
    - The **constraint** is that the larger disk can never be placed on top of a smaller disk.
- ➔ • **Only one disk** can be moved at each time



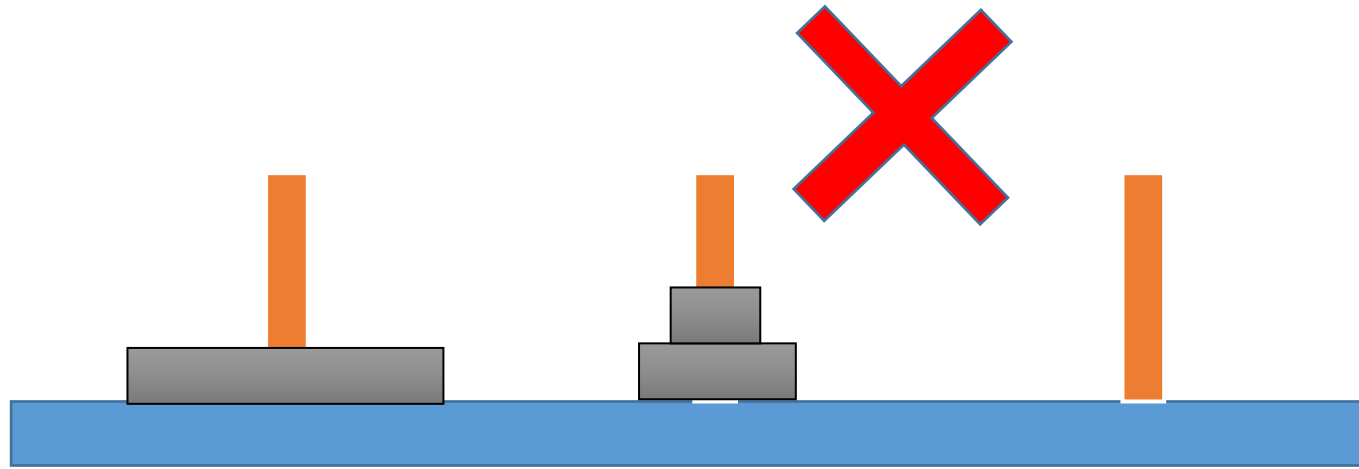
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
    - Assume there are **three towers available**
    - The **constraint** is that the larger disk can never be placed on top of a smaller disk.
- ➔ • **Only one disk** can be moved at each time



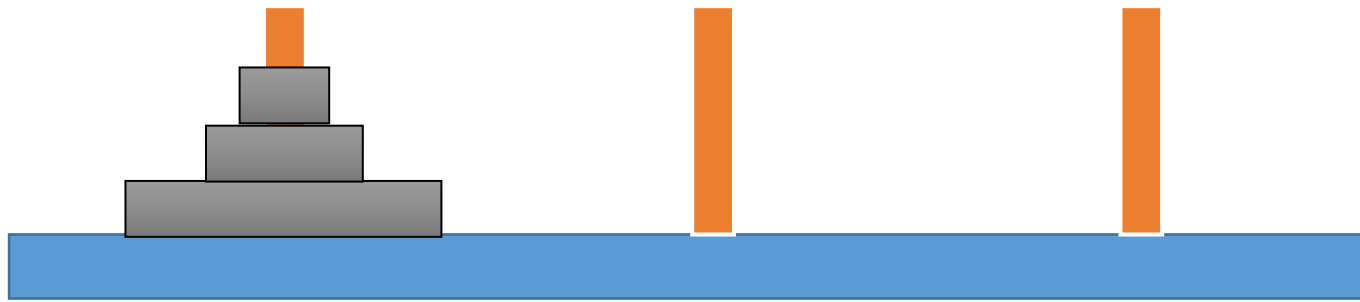
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
  - Assume there are **three towers available**
  - The **constraint** is that the larger disk can never be placed on top of a smaller disk.
- ➔ • **Only one disk** can be moved at each time



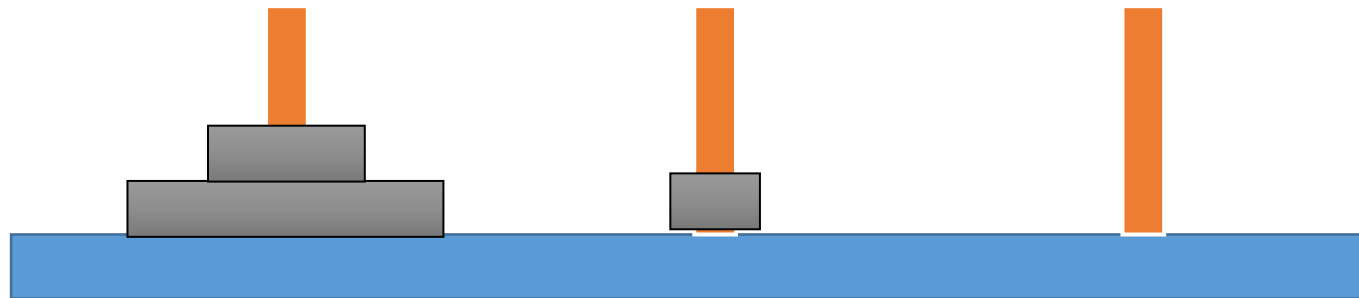
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
    - Assume there are **three towers available**
    - The **constraint** is that the larger disk can never be placed on top of a smaller disk.
- ➔ • **Only one disk** can be moved at each time



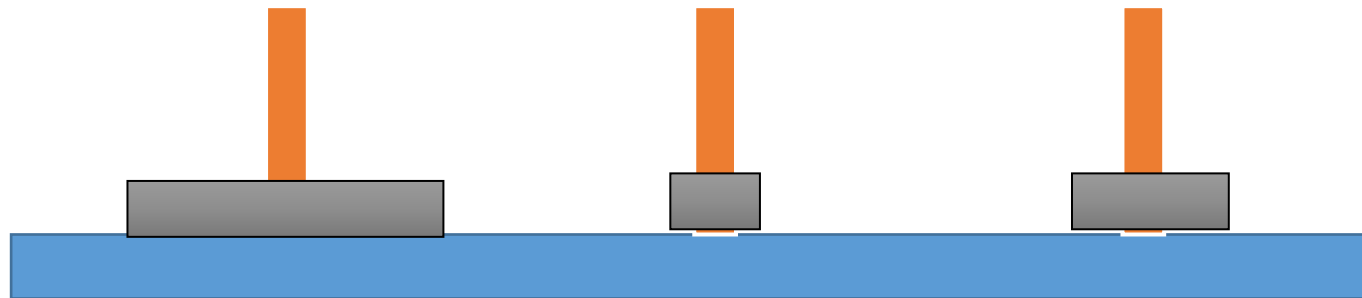
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
  - Assume there are **three towers available**
  - The **constraint** is that the larger disk can never be placed on top of a smaller disk.
- ➔ • **Only one disk** can be moved at each time



# Tower of Hanoi Problem

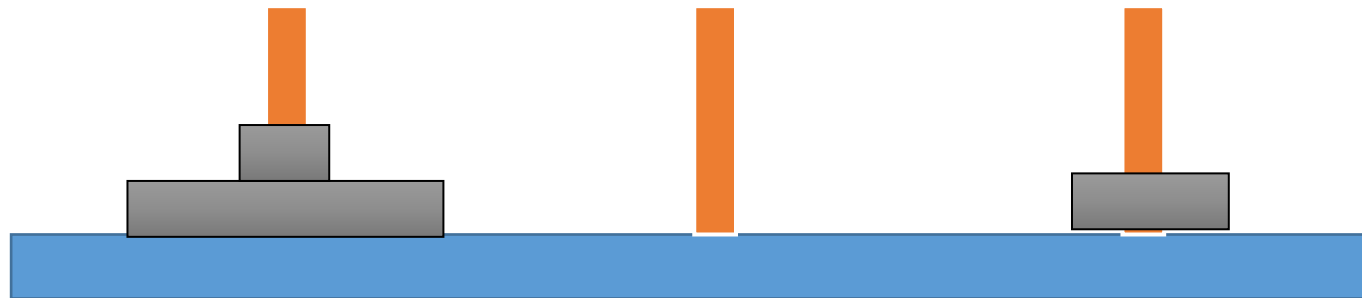
- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
    - Assume there are **three towers available**
    - The **constraint** is that the larger disk can never be placed on top of a smaller disk.
- ➔ • **Only one disk** can be moved at each time





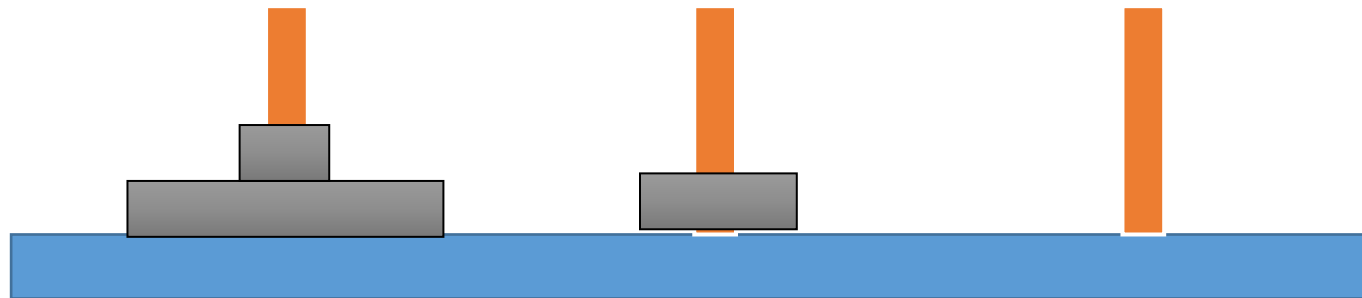
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
    - Assume there are **three towers available**
    - The **constraint** is that the larger disk can never be placed on top of a smaller disk.
- ➔ • **Only one disk** can be moved at each time



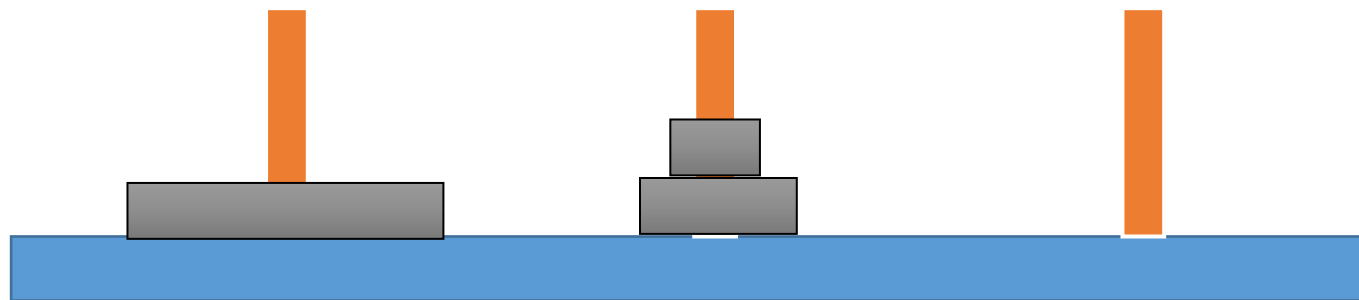
# Tower of Hanoi Problem

- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
    - Assume there are **three towers available**
    - The **constraint** is that the larger disk can never be placed on top of a smaller disk.
- ➔ • **Only one disk** can be moved at each time



# Tower of Hanoi Problem

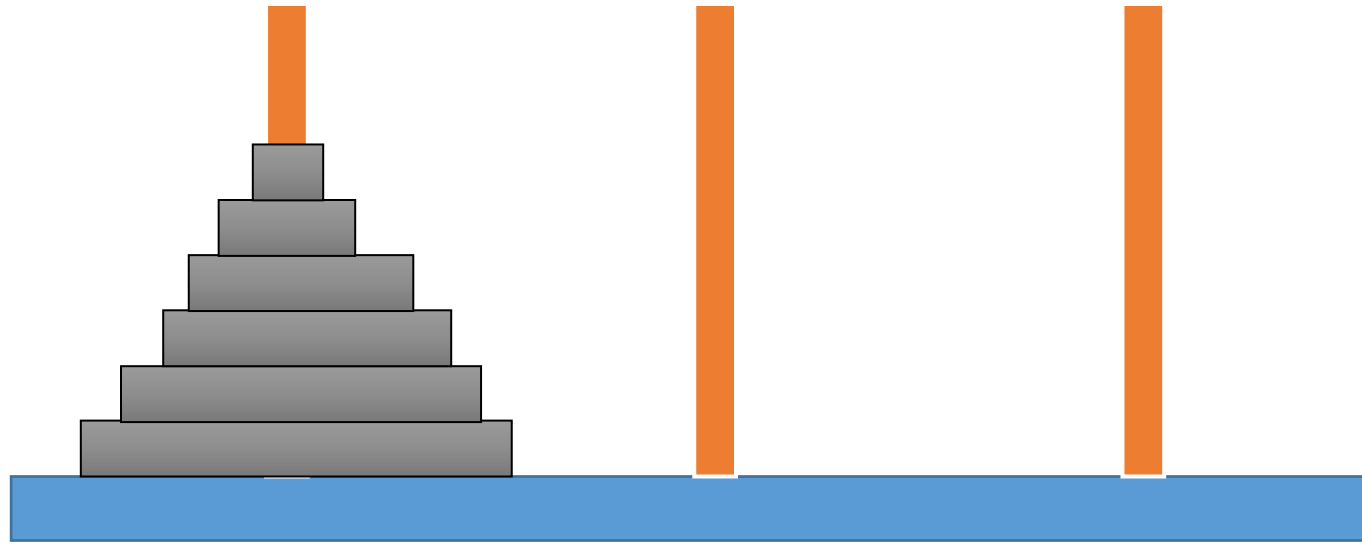
- The towers of Hanoi problem involves moving a number of disks (in different sizes) from one tower (or called “peg”) to another.
  - Assume there are **three towers available**
  - The **constraint** is that the larger disk can never be placed on top of a smaller disk.
- ➔ • **Only one disk** can be moved at each time



OBSERVE THAT WHILE  
TRANSFERRING, WE NEED  
THE HELP OF A THIRD  
TOWER

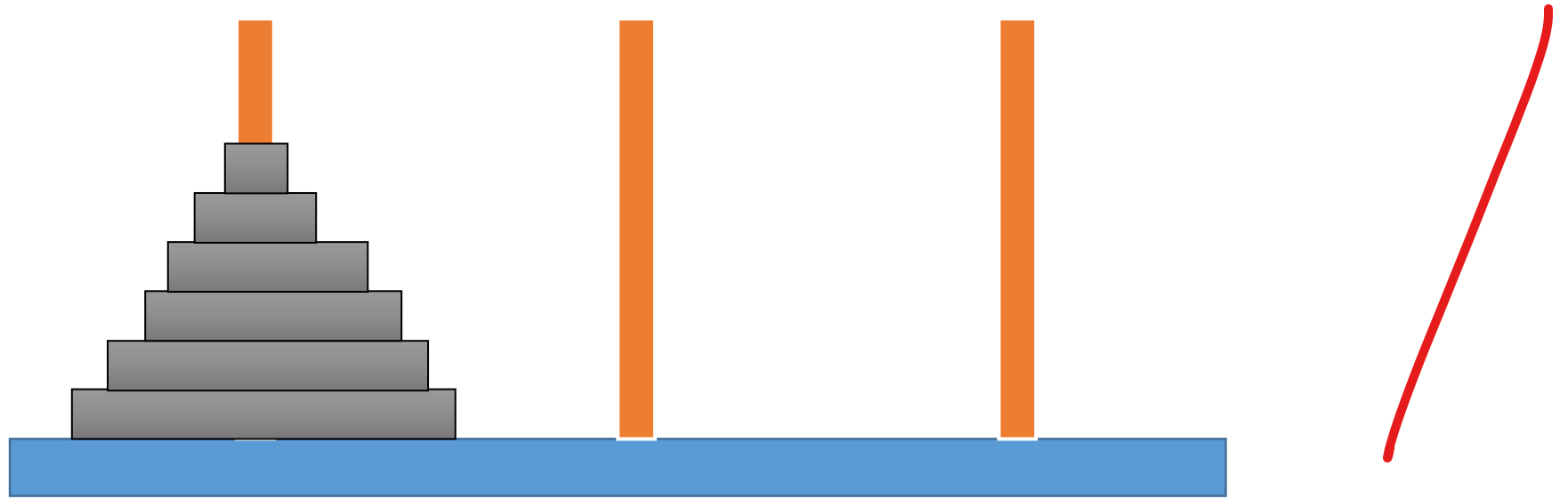
# Tower of Hanoi Problem

- Note that though there are only 3 towers available.
- There can be SEVERAL Disks.



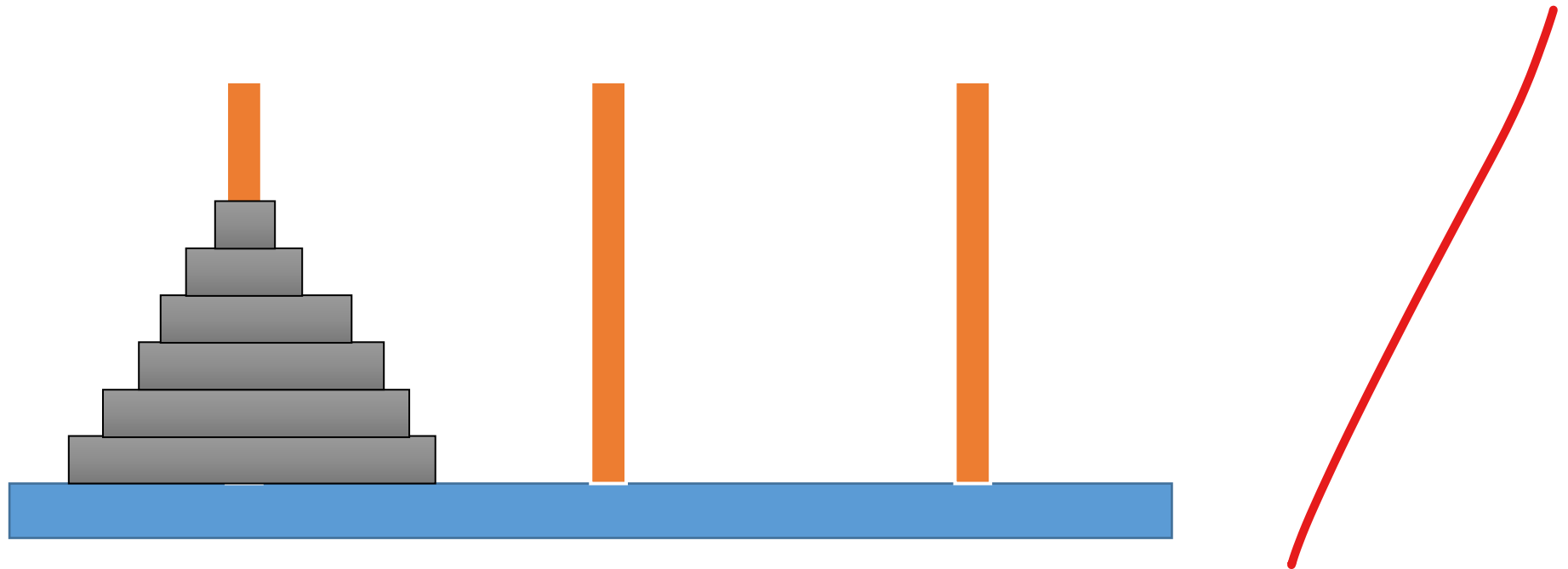
# Tower of Hanoi Problem

- Note that though there are only 3 towers available.
- There can be SEVERAL Disks.



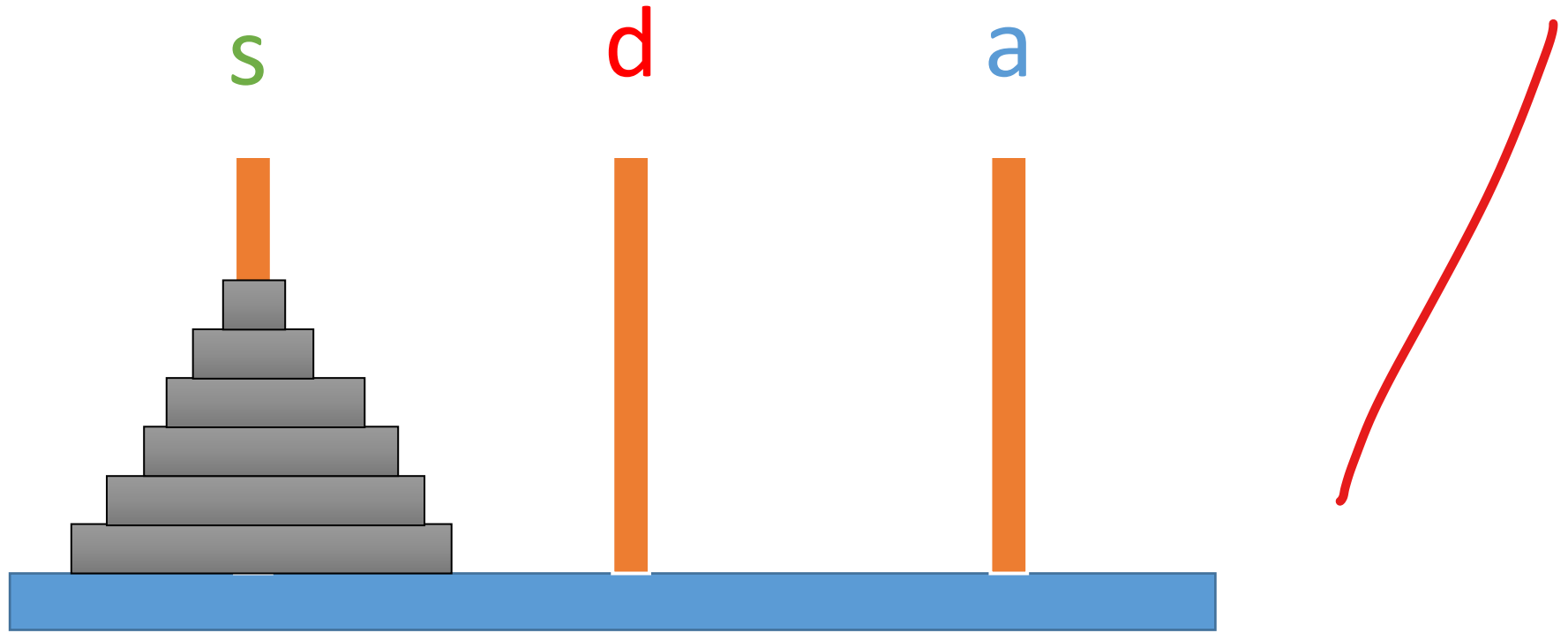
# Tower of Hanoi Problem

- Let the first tower with all discs be **s (source)**. Let the tower where we want to transfer all the discs be **d (destination)**. Let the tower which we will take help of be **a (auxillary)**.



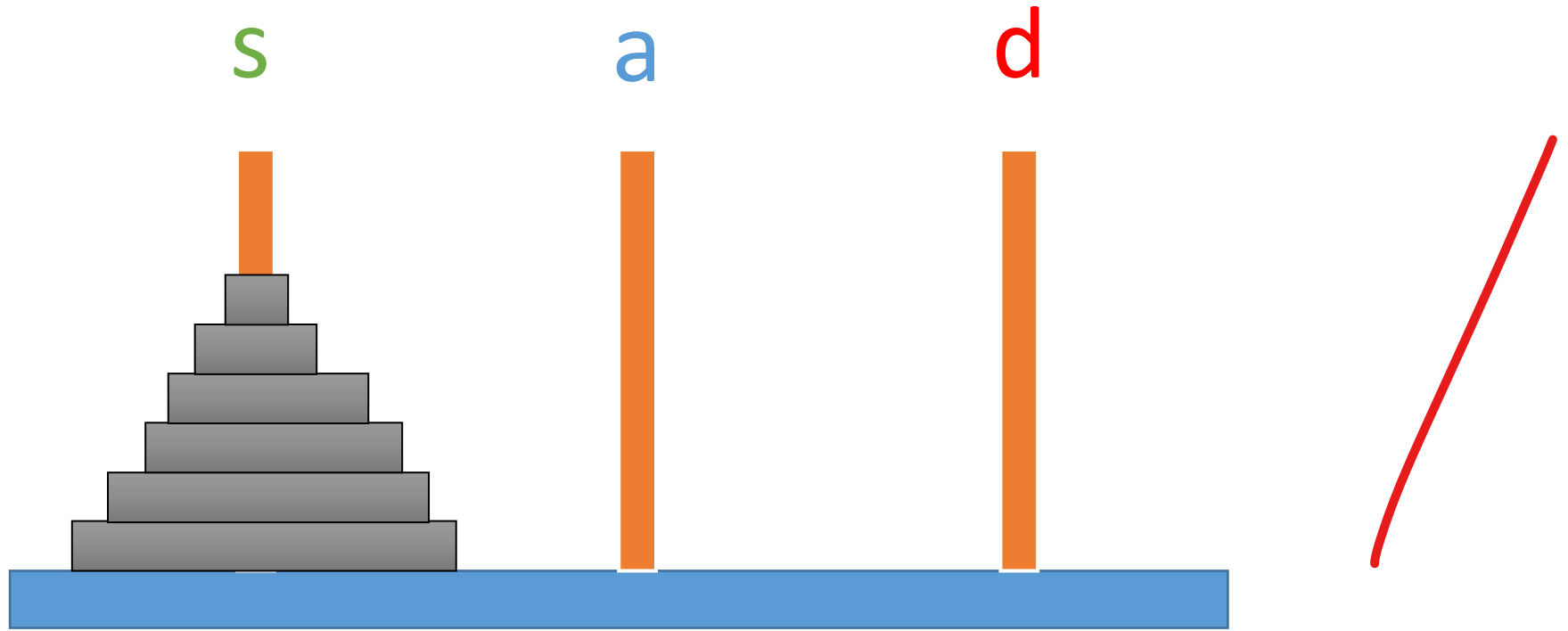
# Tower of Hanoi Problem

- Let the first tower with all discs be **s (source)**. Let the tower where we want to transfer all the discs be **d (destination)**. Let the tower which we will take help of be **a (auxiliary)**.



# Tower of Hanoi Problem

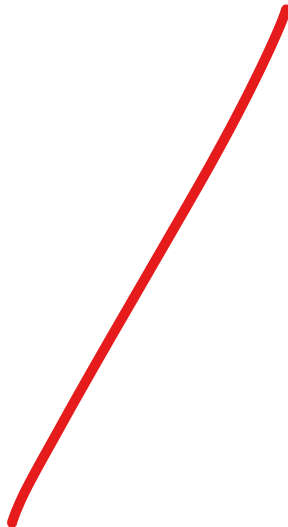
- Let the first tower with all discs be **s (source)**. Let the tower where we want to transfer all the discs be **d (destination)**. Let the tower which we will take help of be **a (auxiliary)**.







# LET US DEVELOP A RECURSIVE ALGORITHM

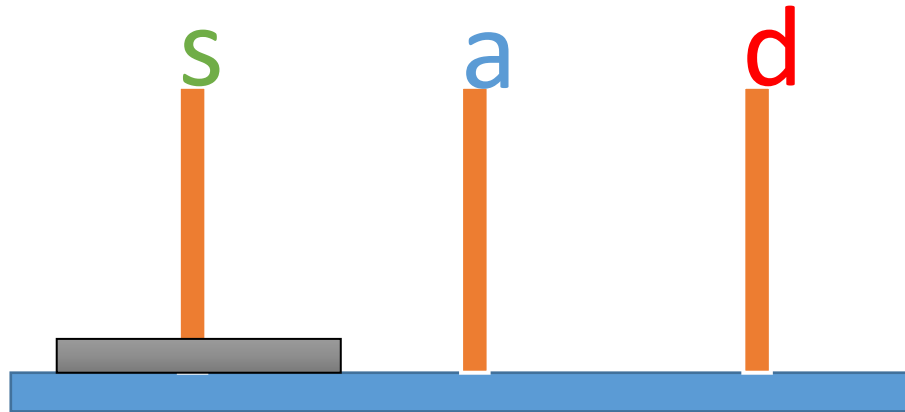


# The Recursive Algorithm

- The algorithm will perform the following tasks:
  - It will **transfer** all the discs from the source to the destination
  - It will print all the moves
    - (transfer of **ONE disk** from the **TOP** of one tower to another)
    - It will also give us the **total number of moves**. (by move we again mean the transfer of one disc)
- Let us consider the following cases.

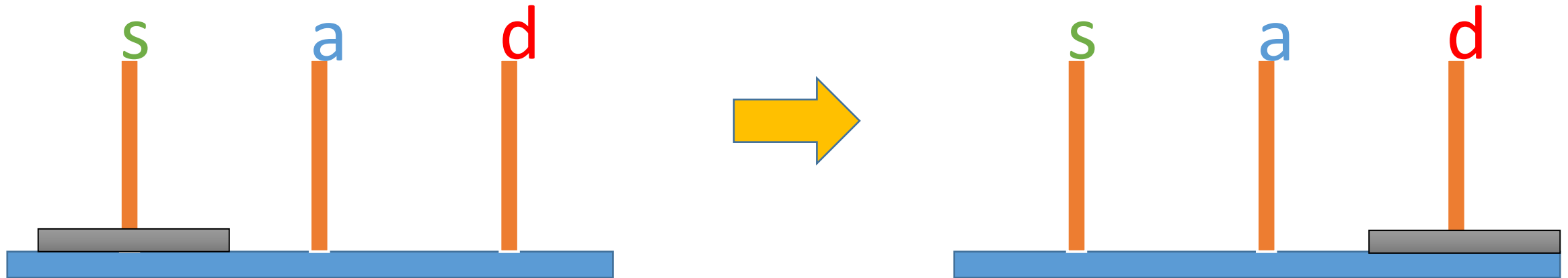
# The Recursive Algorithm

There is only one disk.



# The Recursive Algorithm

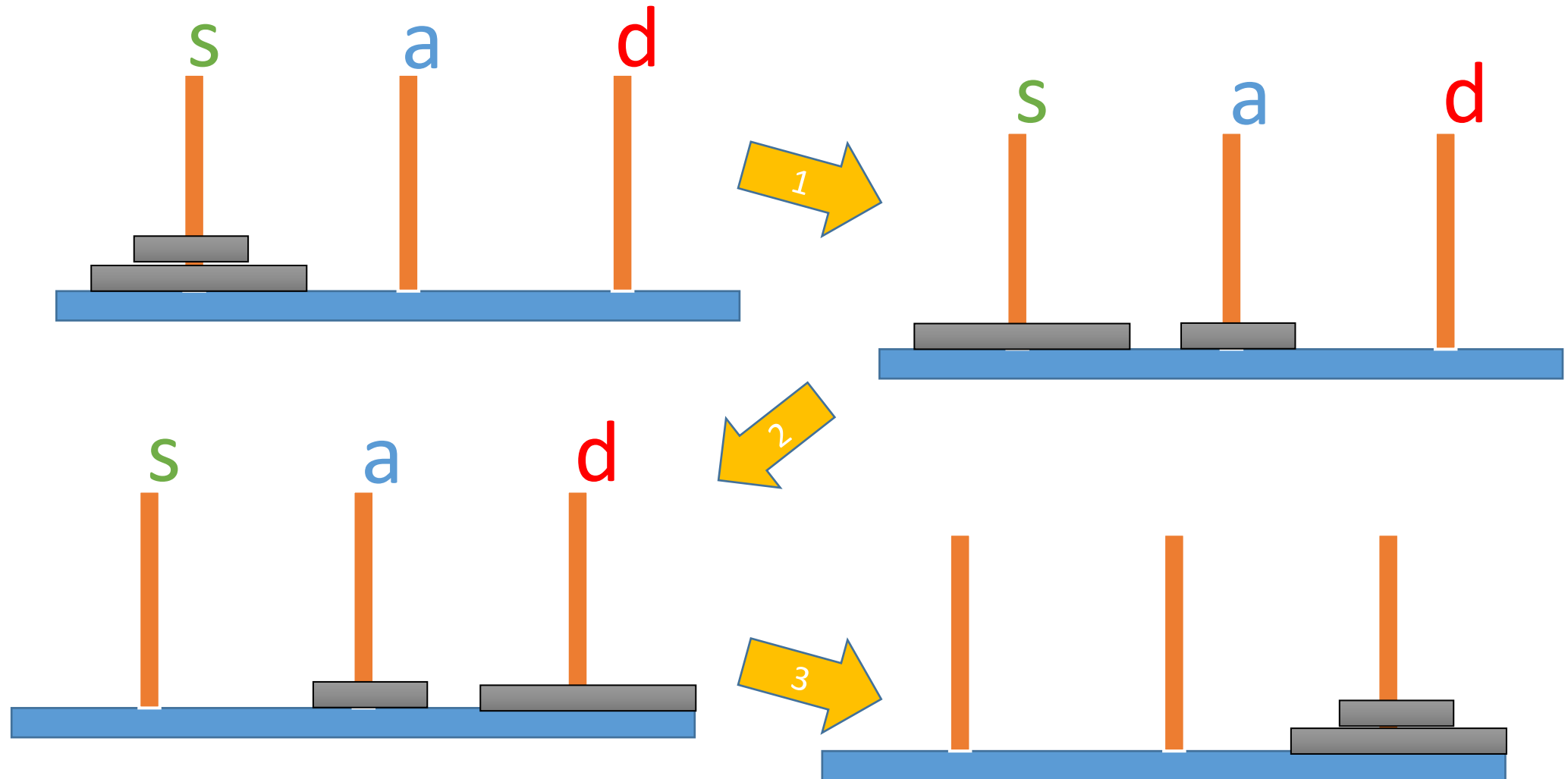
There is only one disk.



# The Recursive Algorithm

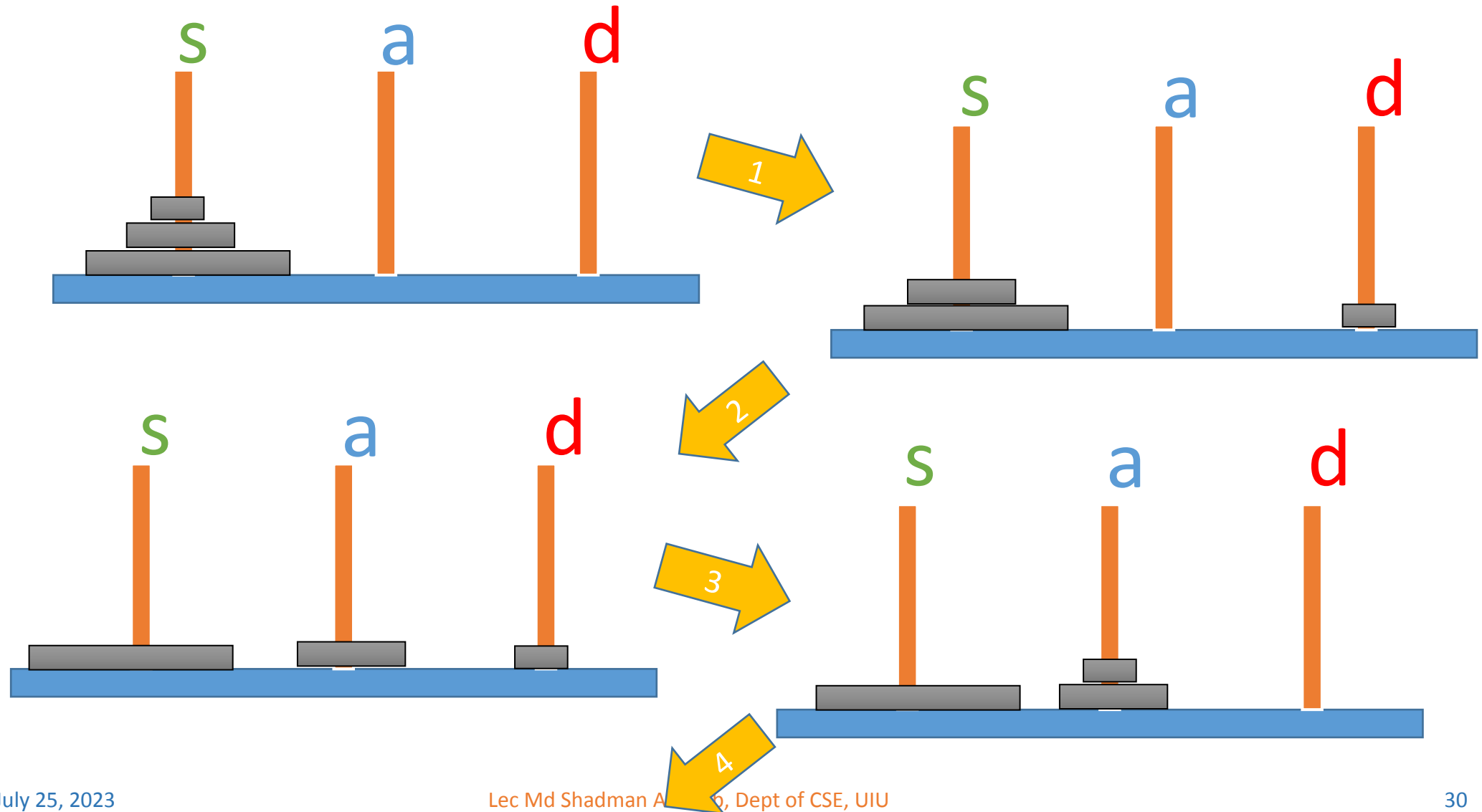
There are two discs

A total of 3 moves.



# The Recursive Algorithm

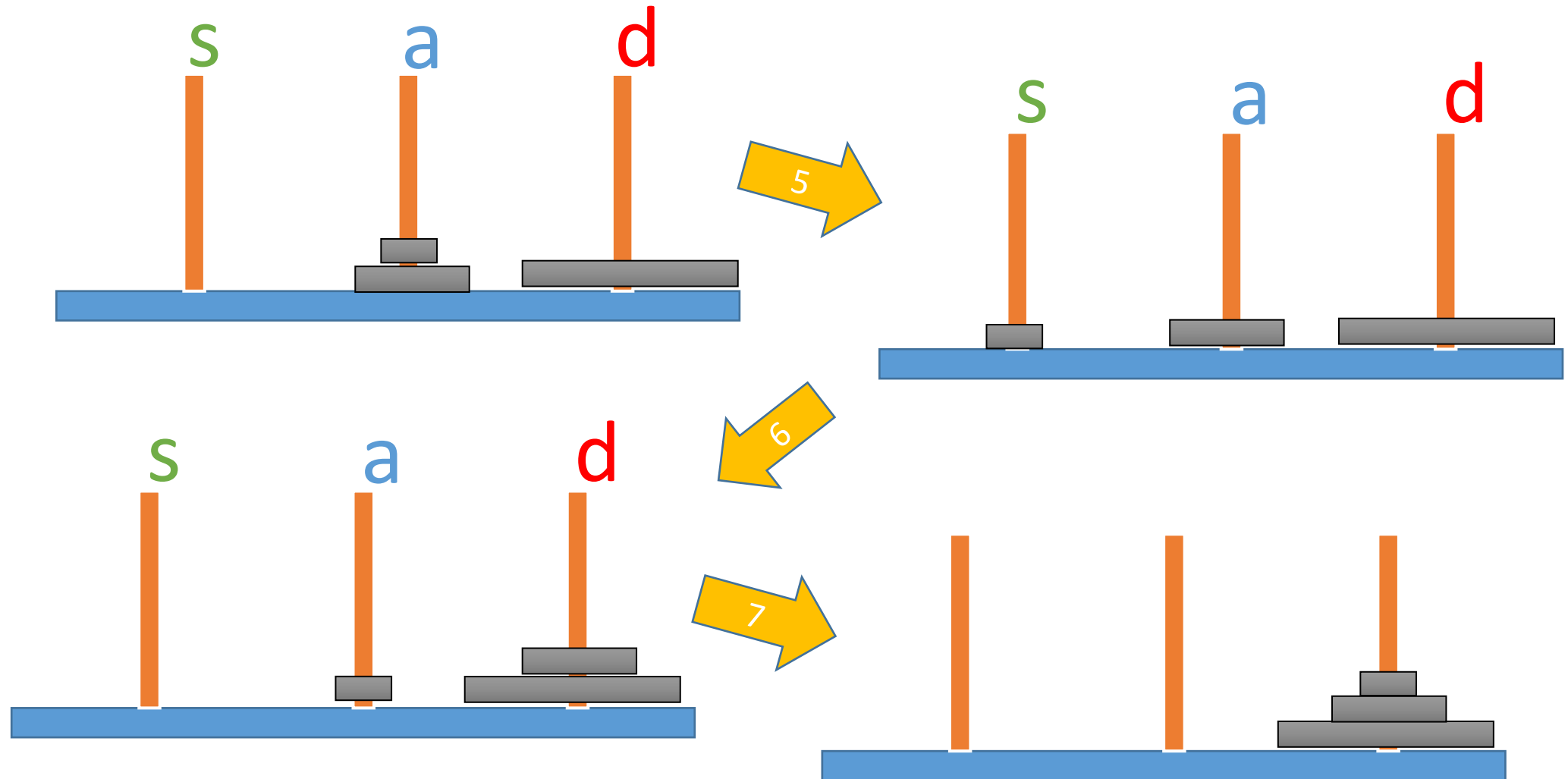
There are three discs



# The Recursive Algorithm

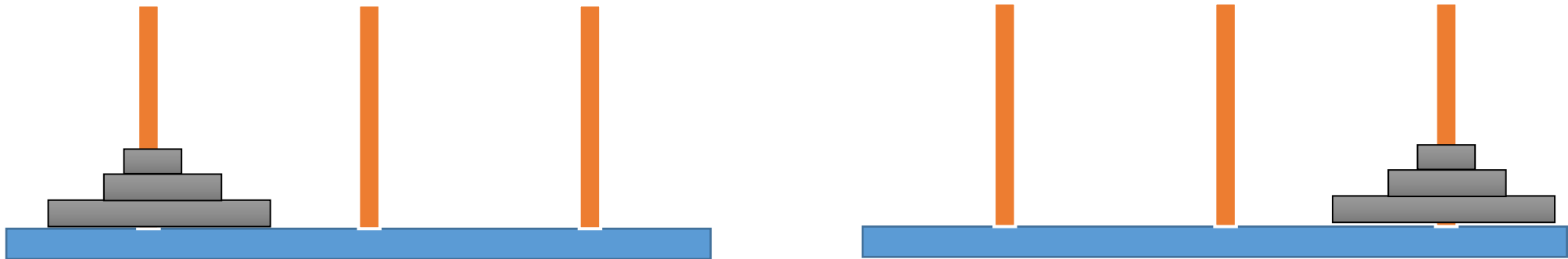
There are three discs

A total of 7 moves.

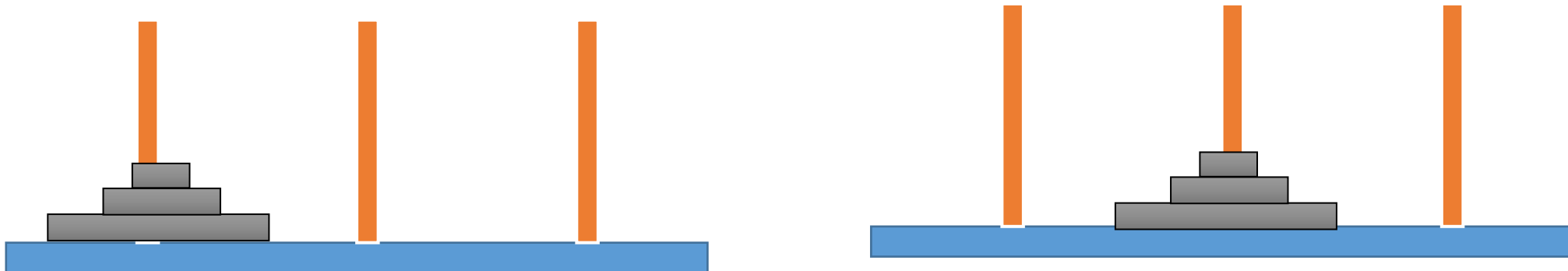


# Observe

- We can transfer all discs from the first tower to the third tower (with the help of the second tower)



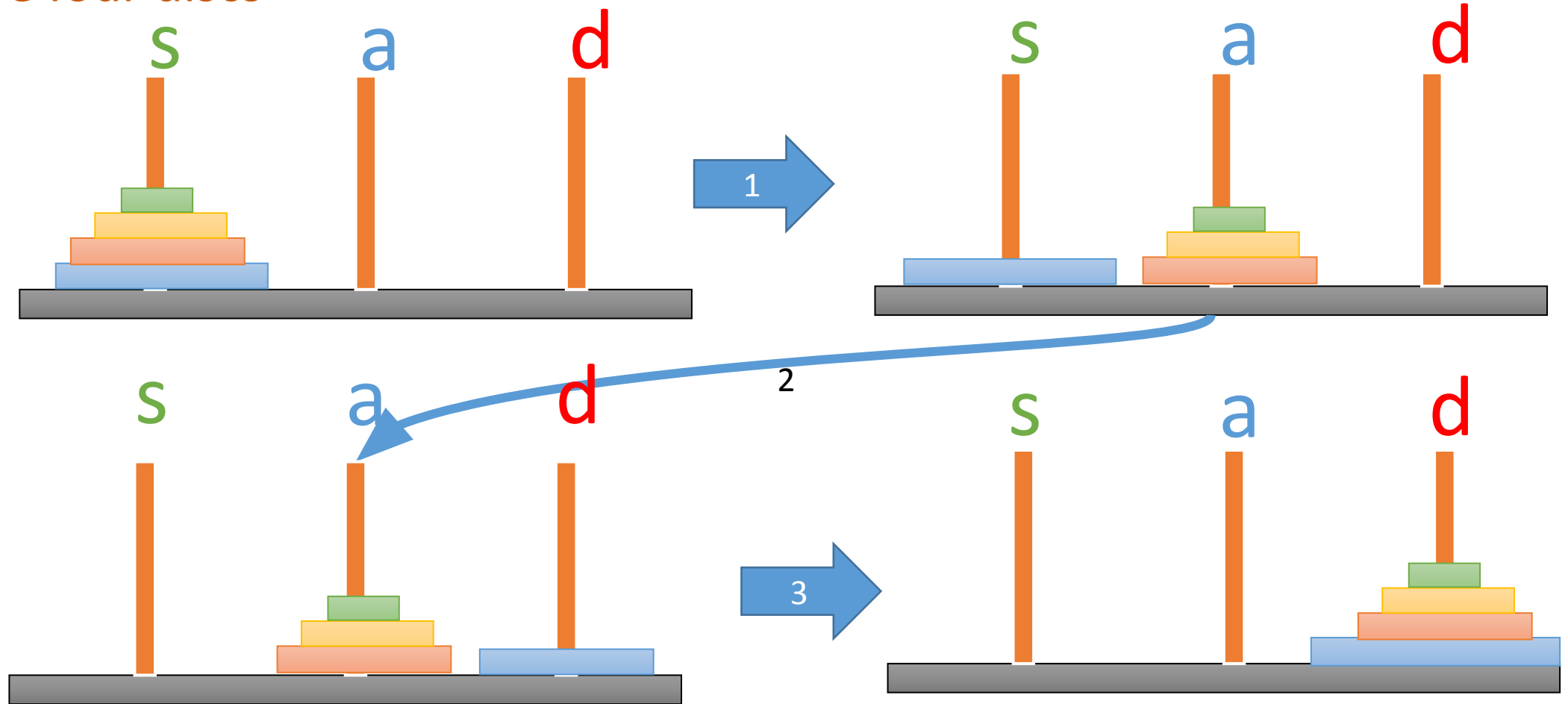
- We can transfer all discs from the first tower to the second tower (with the help of the third tower)





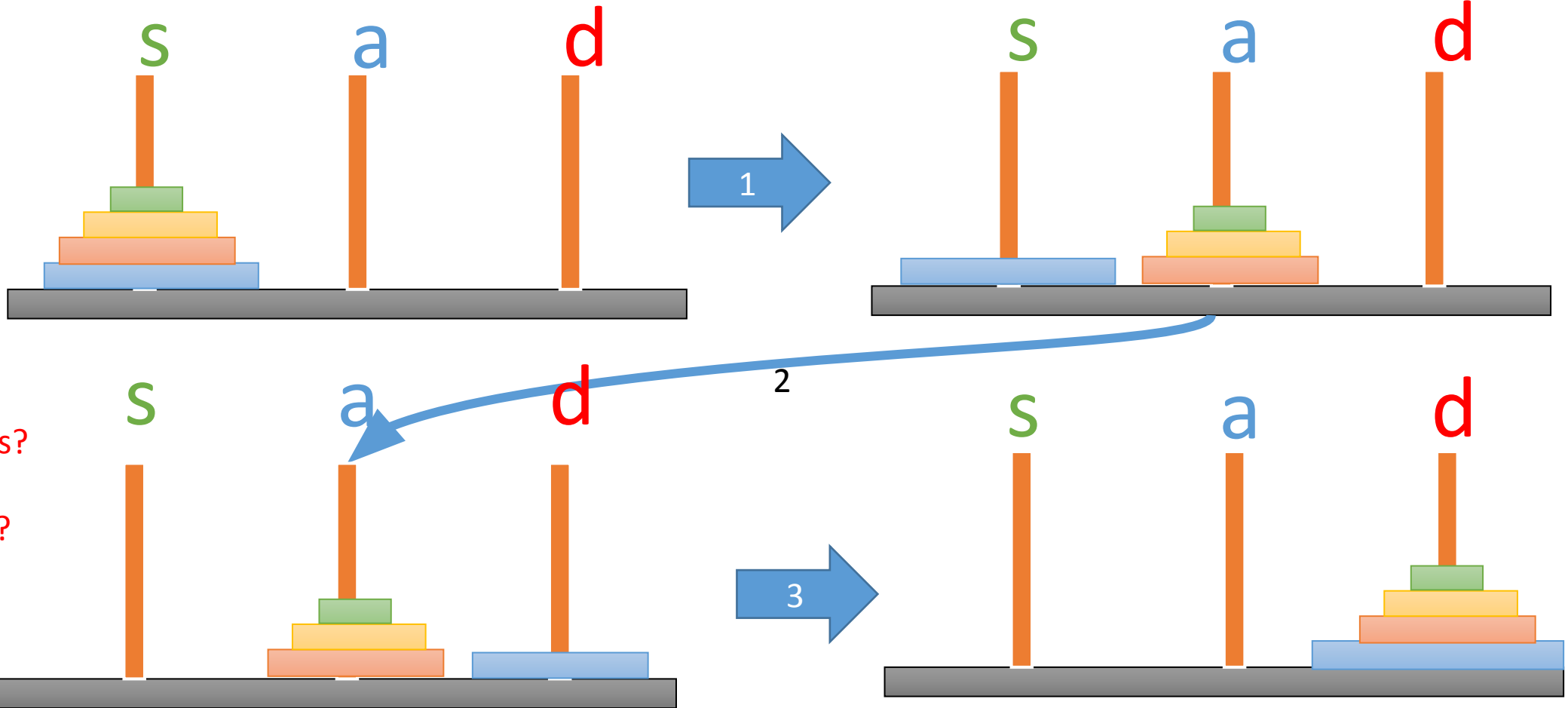
# The Recursive Algorithm

There are four discs



# The Recursive Algorithm

There are four discs



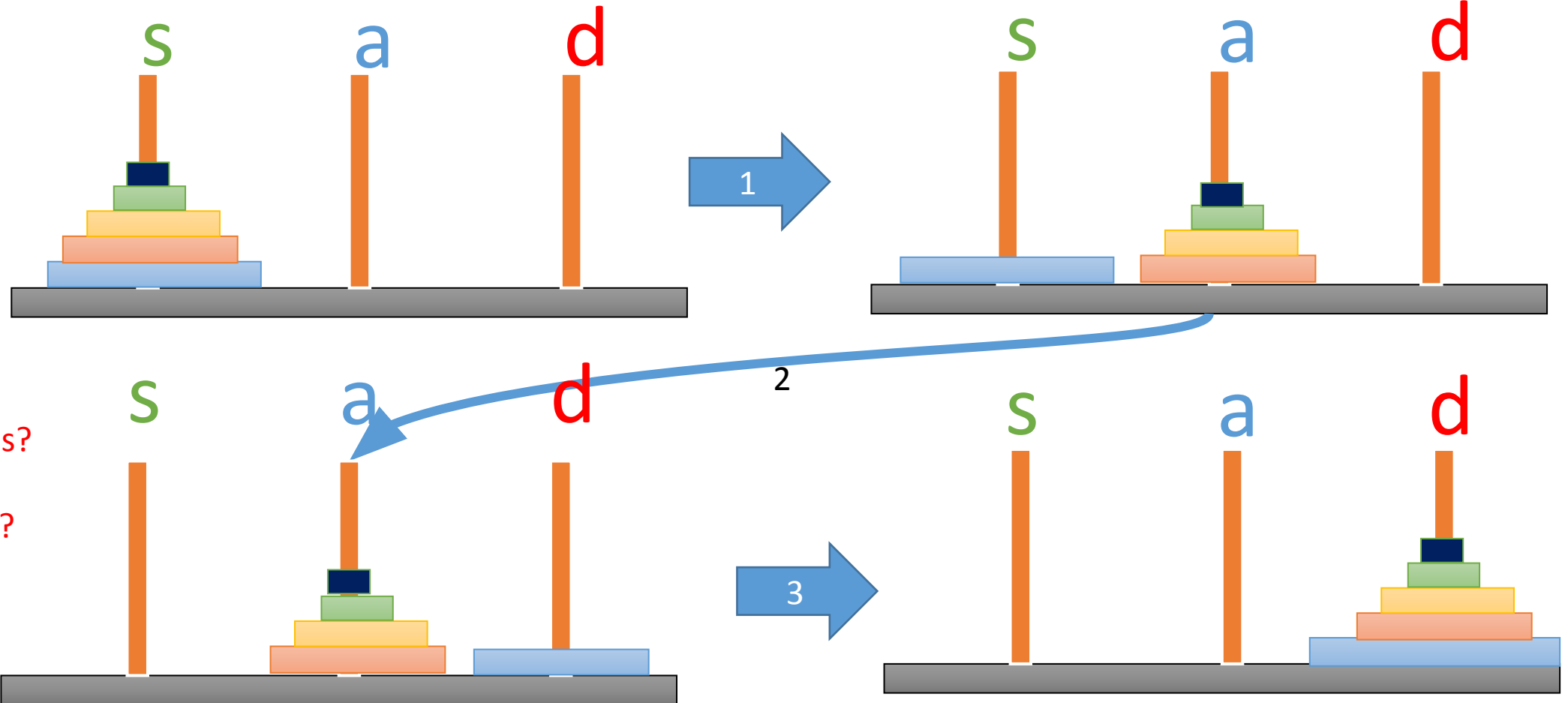
What are the steps?

How many moves?

$$7+1+7=15$$

# The Recursive Algorithm

There are five discs



What are the steps?

How many moves?

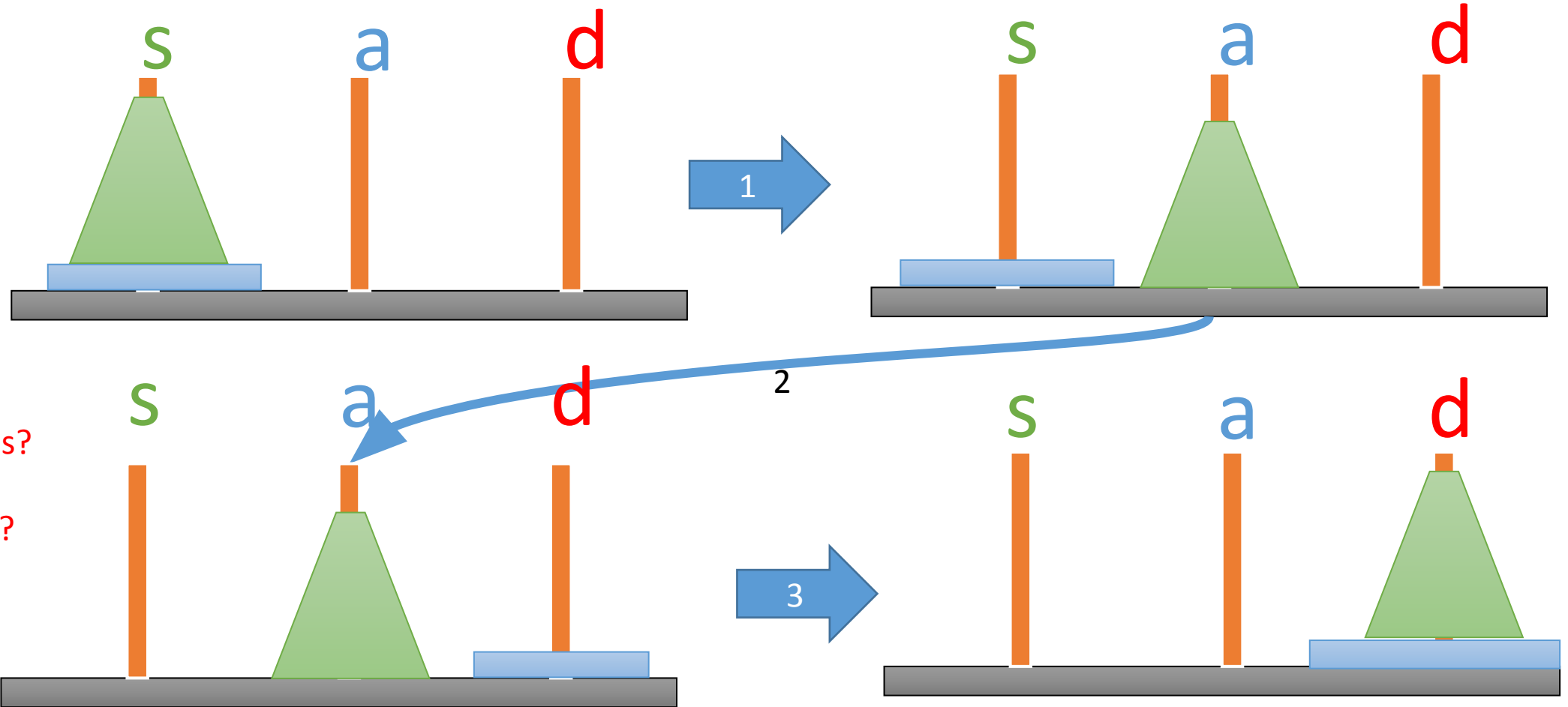
$$15+1+15=31$$

# The Recursive Algorithm

- We can transfer 3 discs -> We can transfer 4 discs
- We can transfer 4 discs -> We can transfer 5 discs
- We can transfer 5 discs -> We can transfer 6 discs
- We can transfer  $n-1$  discs -> We can transfer  $n$  discs

# The Recursive Algorithm

For n number of disks

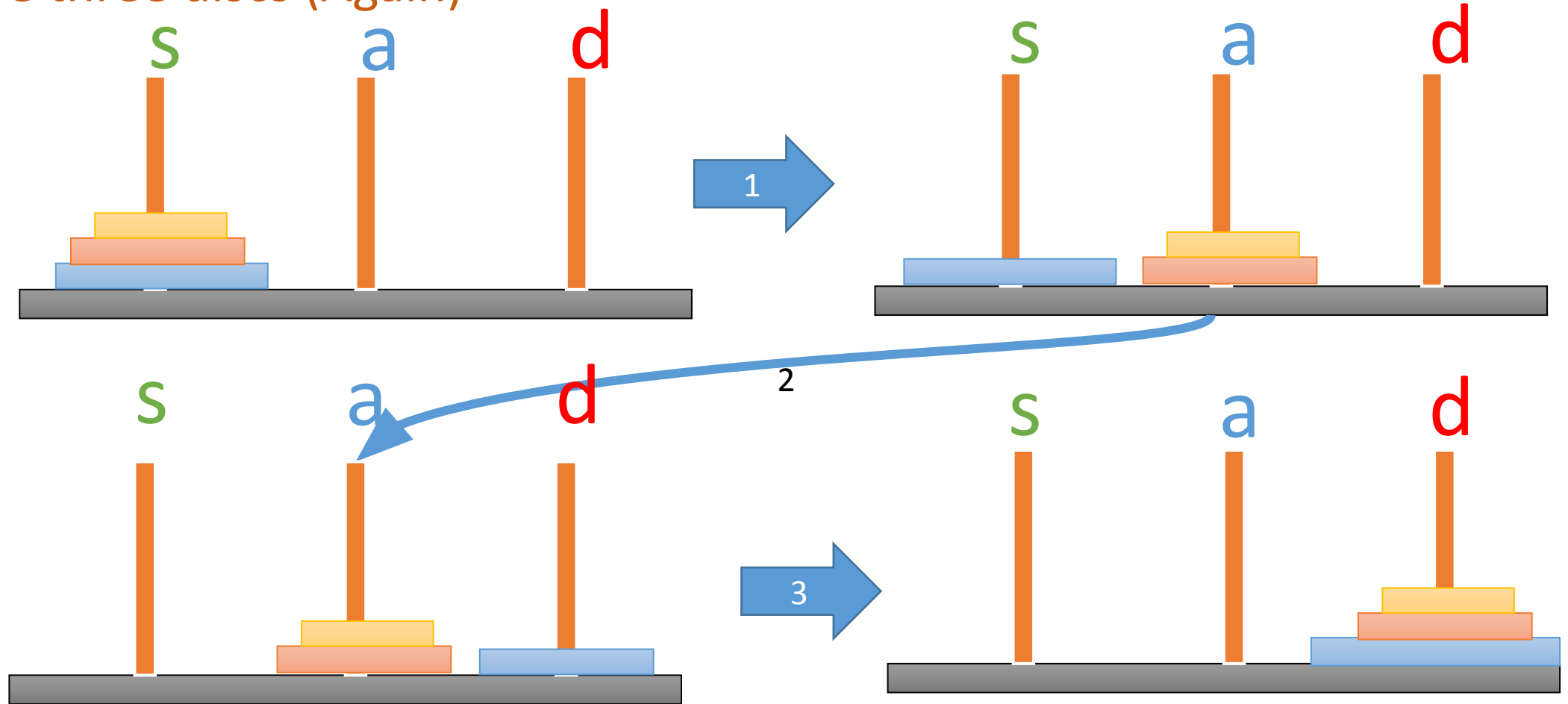


What are the steps?

How many moves?

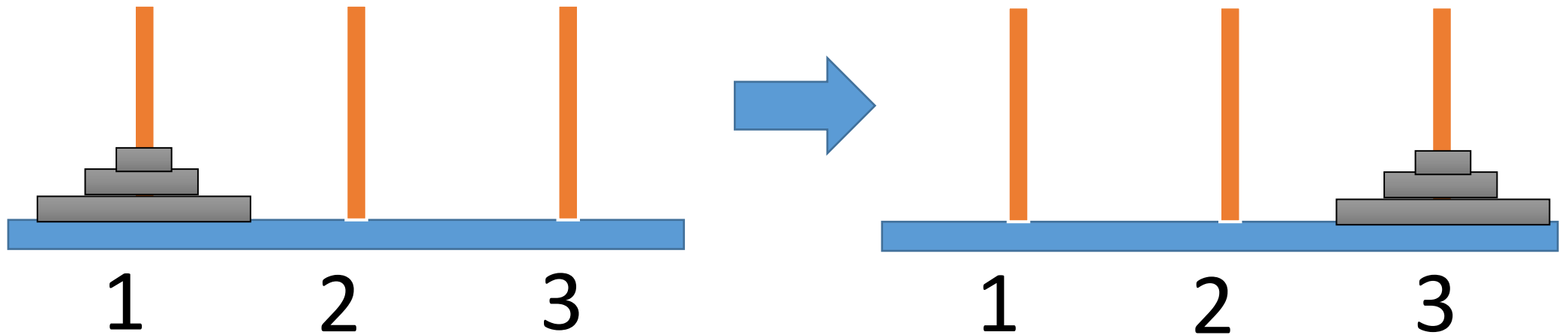
# The Recursive Algorithm

There are three discs (Again)



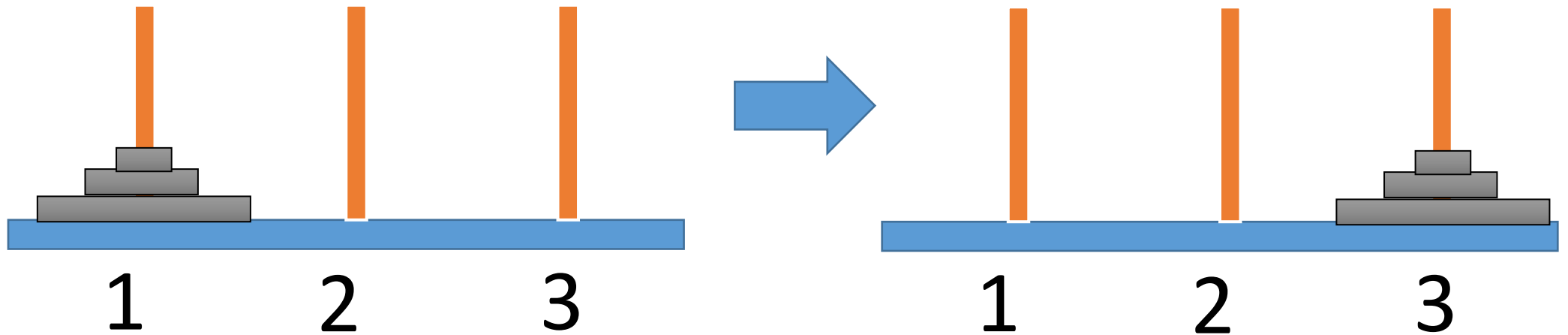
# Function Inputs

- `int Hanoi (int n, int s, int d, int a)`
- Calling the function:



# Function Inputs

- `int Hanoi (int n, int s, int d, int a)`
- Calling the function:
  - `x=Hanoi (3, 1, 3, 2)`





# Function Inputs

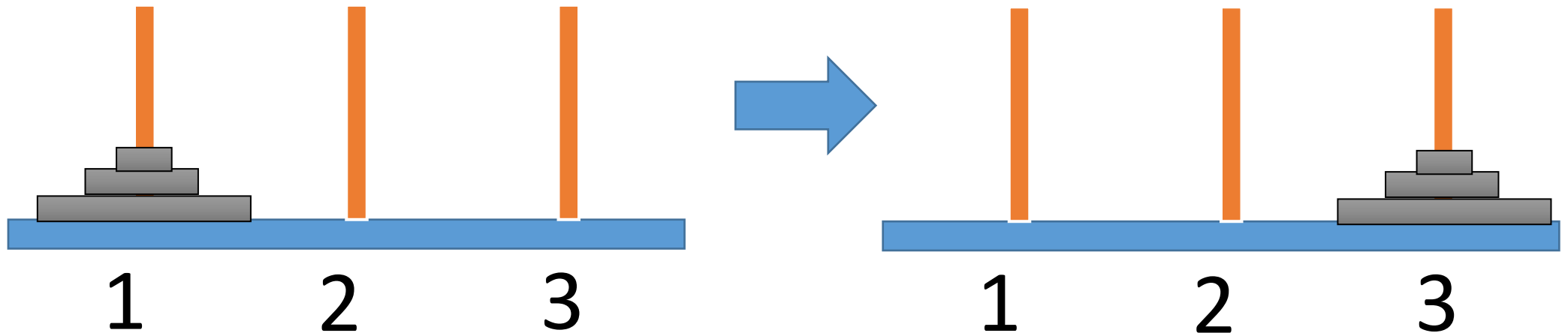
- `int Hanoi (int n, int s, int d, int a)`

- Calling the function:

- `x=Hanoi (3, 1, 3, 2)`

How many steps are needed (and what are the steps)?

To move 3 discs



# Function Inputs

- `int Hanoi (int n, int s, int d, int a)`

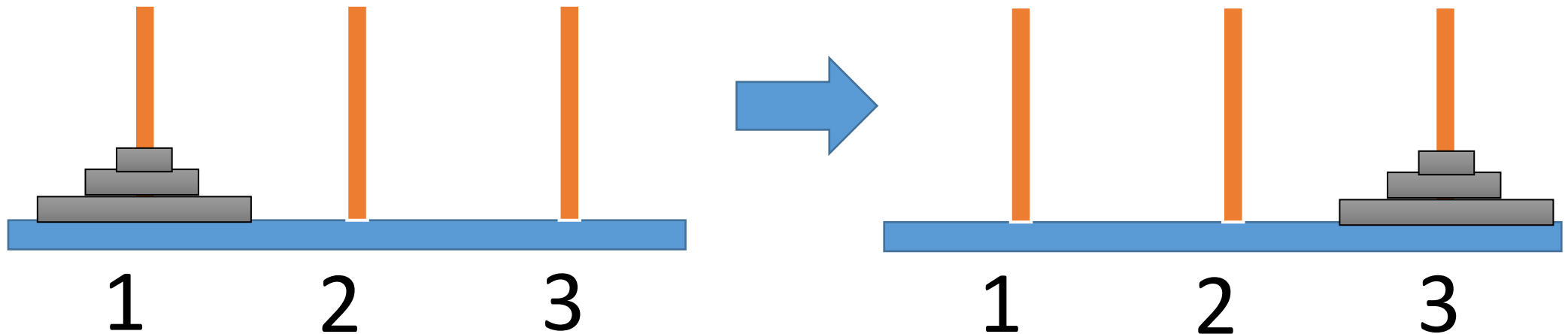
- Calling the function:

- `x=Hanoi (3, 1, 3, 2)`

How many steps are needed (and what are the steps)?

To Move 3 discs

From tower 1



# Function Inputs

- `int Hanoi (int n, int s, int d, int a)`

- Calling the function:

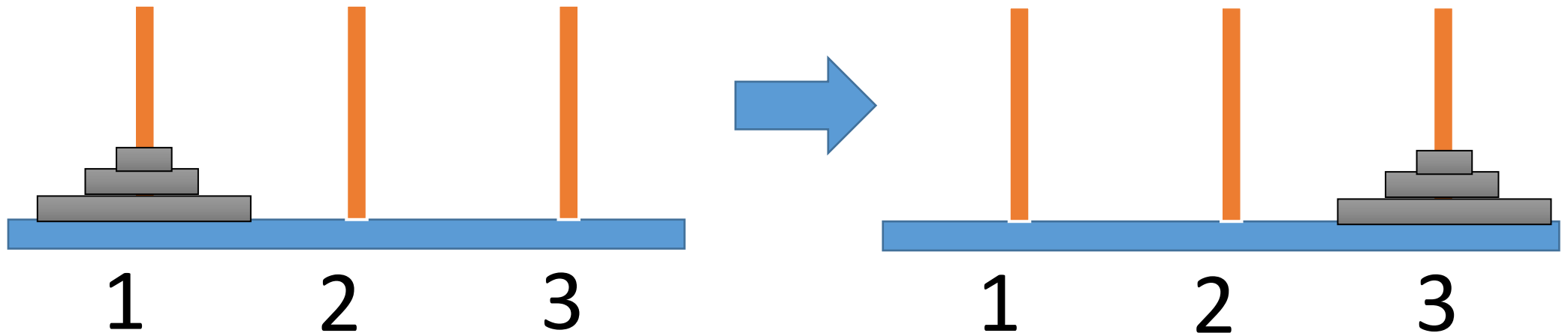
- `x=Hanoi (3, 1, 3, 2)`

How many steps are needed (and what are the steps)?

To Move 3 discs

From tower 1

To tower 3



# Function Inputs

- `int Hanoi (int n, int s, int d, int a)`

- Calling the function:

- `x=Hanoi (3, 1, 3, 2)`

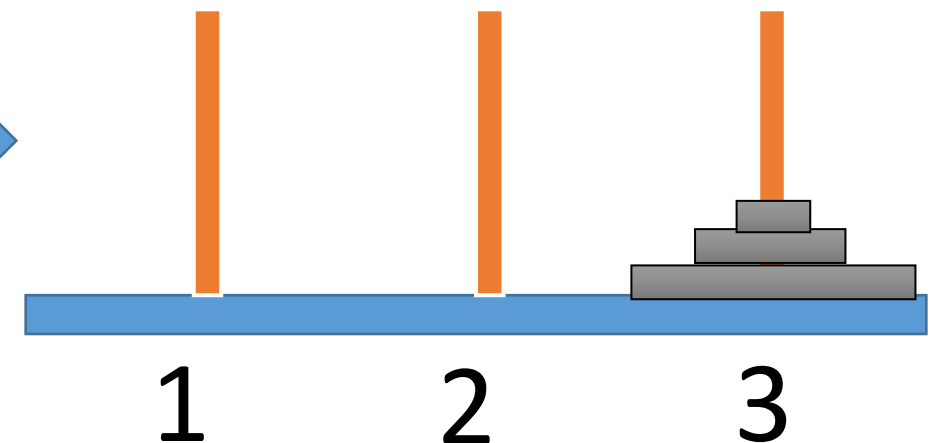
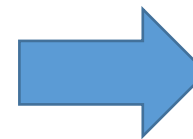
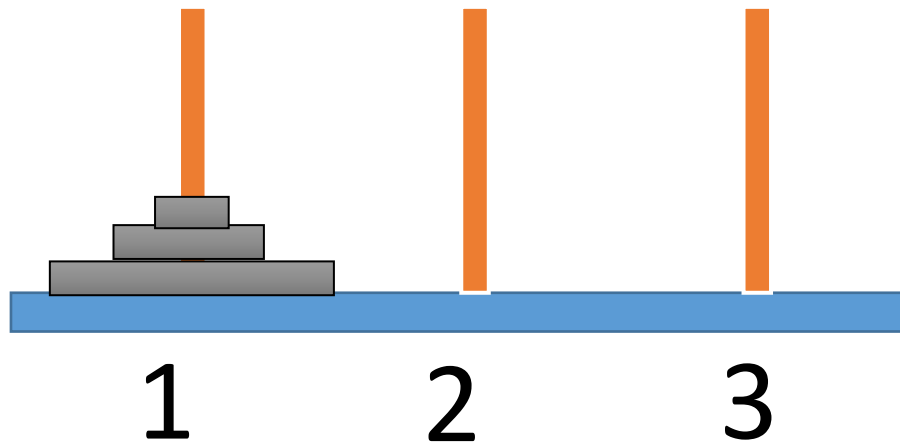
How many steps are needed (and what are the steps)?

To Move 3 discs

From tower 1

To tower 3

By taking the help of tower 2



# Function Inputs

- `int Hanoi (int n, int s, int d, int a)`

- Calling the function:

- `x = Hanoi (3, 1, 2, 3)`

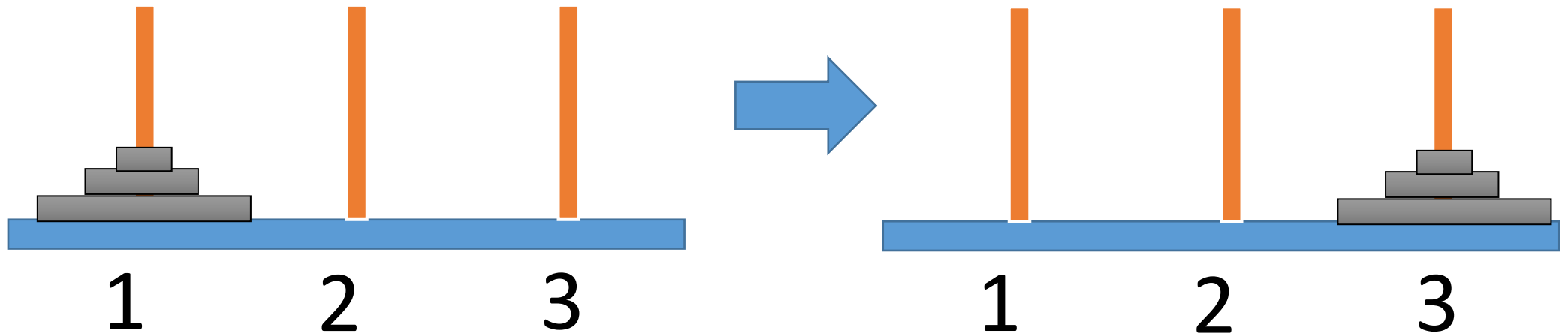
How many steps are needed (and what are the steps)?

To Move 3 discs

From tower 1

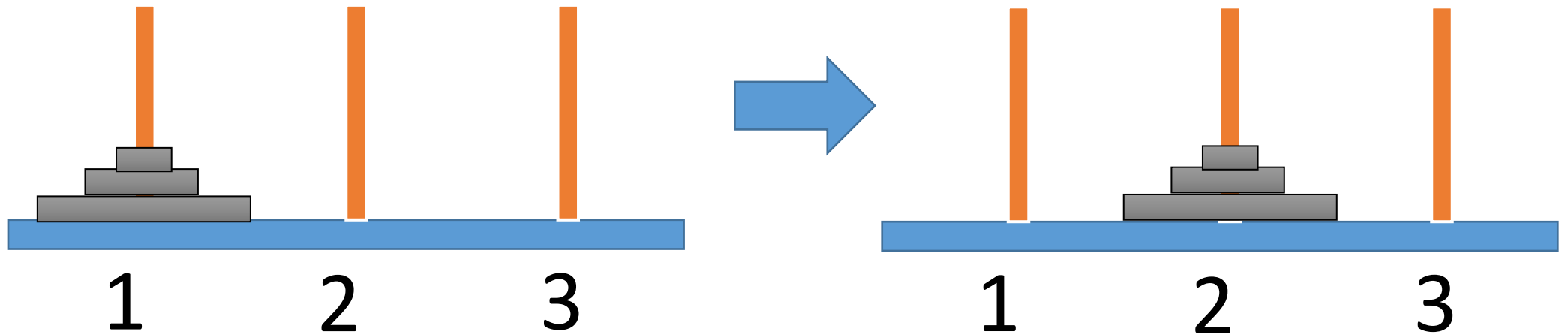
To tower 3

By taking the help of tower 2



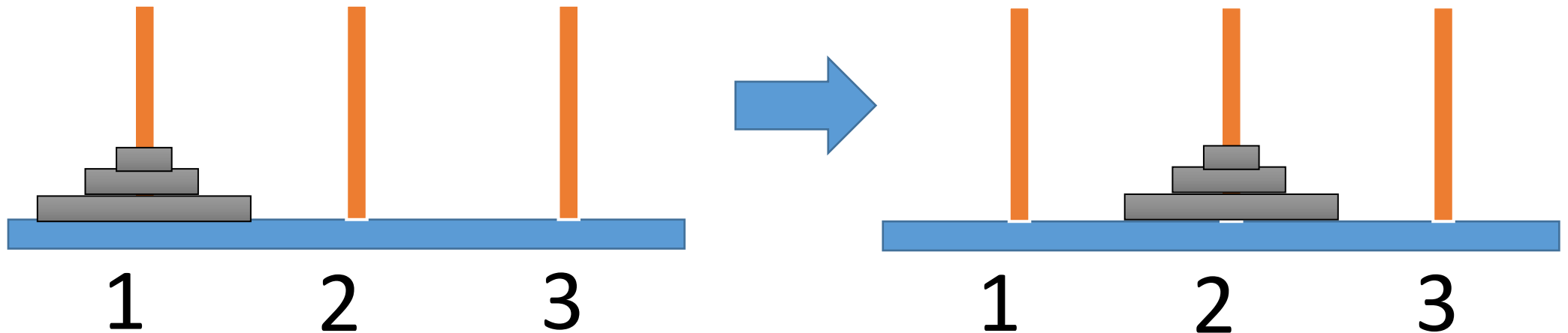
# Function Inputs

- `int Hanoi (int n, int s, int d, int a)`
- How will you call the function in this case?



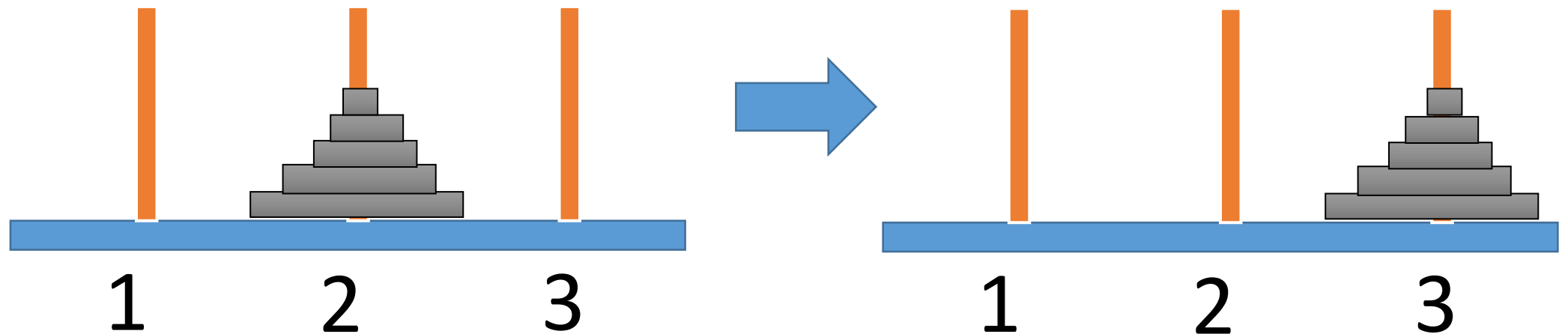
# Function Inputs

- `int Hanoi (int n, int s, int d, int a)`
- How will you call the function in this case?
- **Ans:** `x=Hanoi (3, 1, 2, 3)`



# Function Inputs

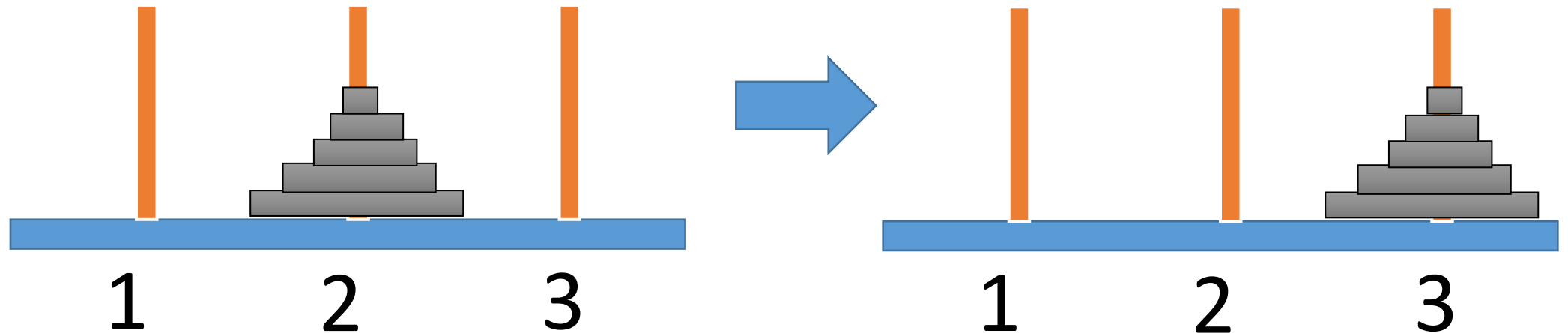
- `int Hanoi (int n, int s, int d, int a)`
- How will you call the function in this case?





# Function Inputs

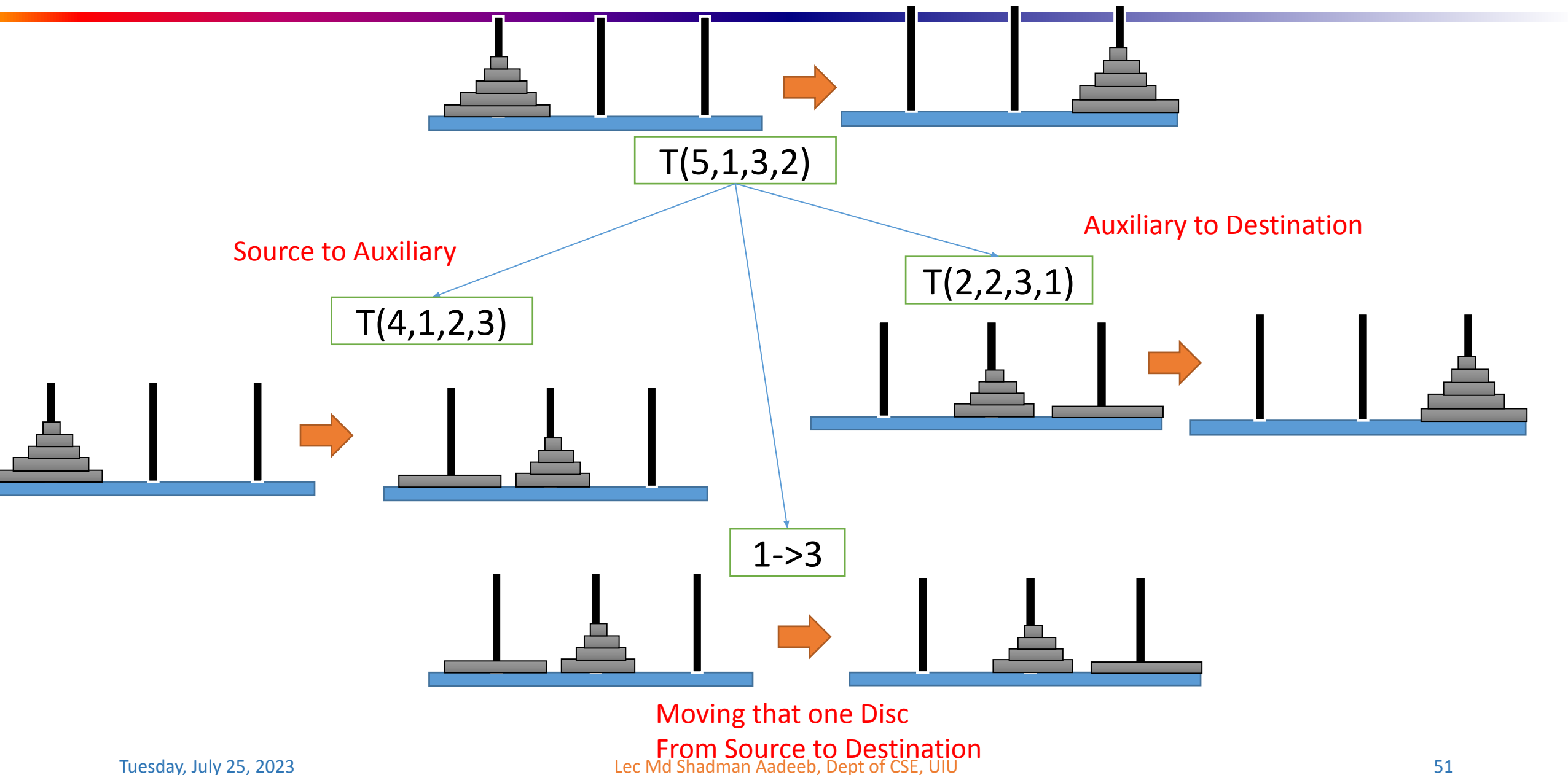
- `int Hanoi (int n, int s, int d, int a)`
- How will you call the function in this case?
- **Ans:** `x=Hanoi (5, 2, 3, 1)`



# Let's Make the Function Work

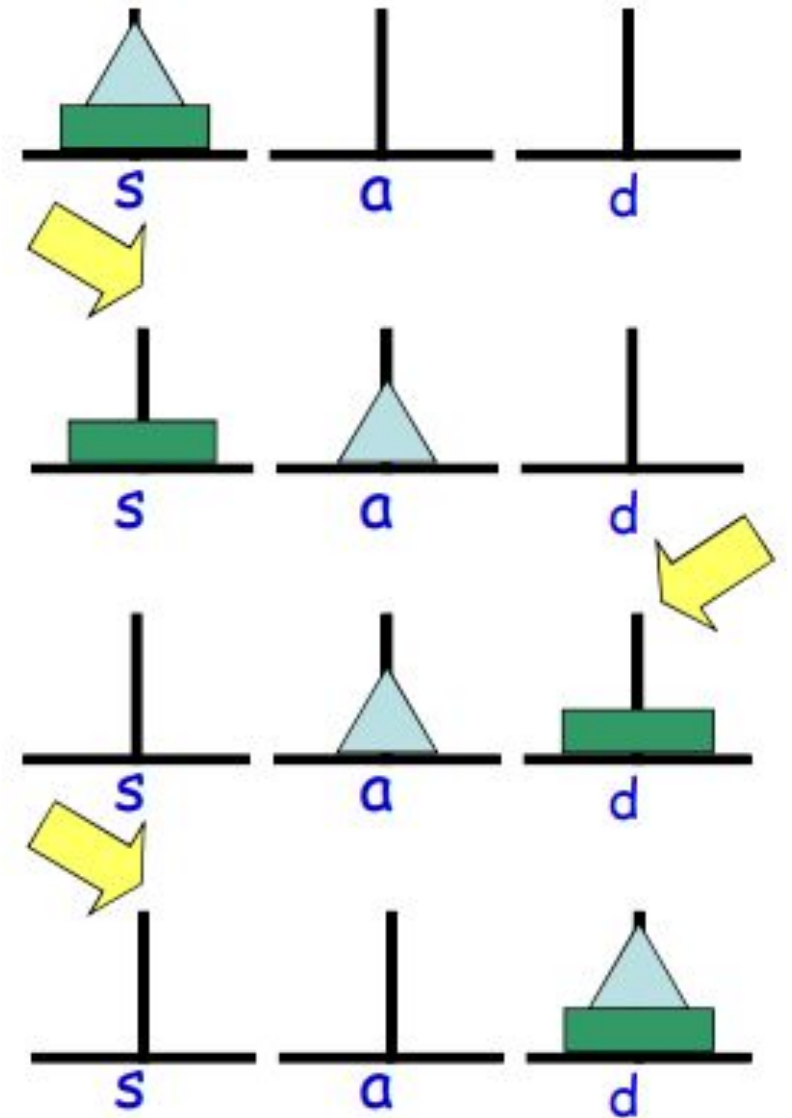
```
int Hanoi(int n, int s, int d, int a) {  
    if (n==1) {  
        print("%d -> %d", s, d);  
        return 1;  
    }  
    else {  
        x1=Hanoi(n-1, s, a, d);  
        print("%d -> %d", s, d); count++;  
        x2=Hanoi(n-1, a, d, s);  
        return x1+x2+1  
    }  
}
```

# Understanding how the recursion is working



# Let's Make the Function Work

```
int Hanoi(int n, int s, int d, int a) {  
    if (n==1) {  
        print("%d -> %d", s, d);  
        return 1;  
    }  
    else {  
        x1=Hanoi(n-1, s, a, d);  
        print("%d -> %d", s, d); count++;  
        x2=Hanoi(n-1, a, d, s);  
        return x1+x2+1  
    }  
}
```

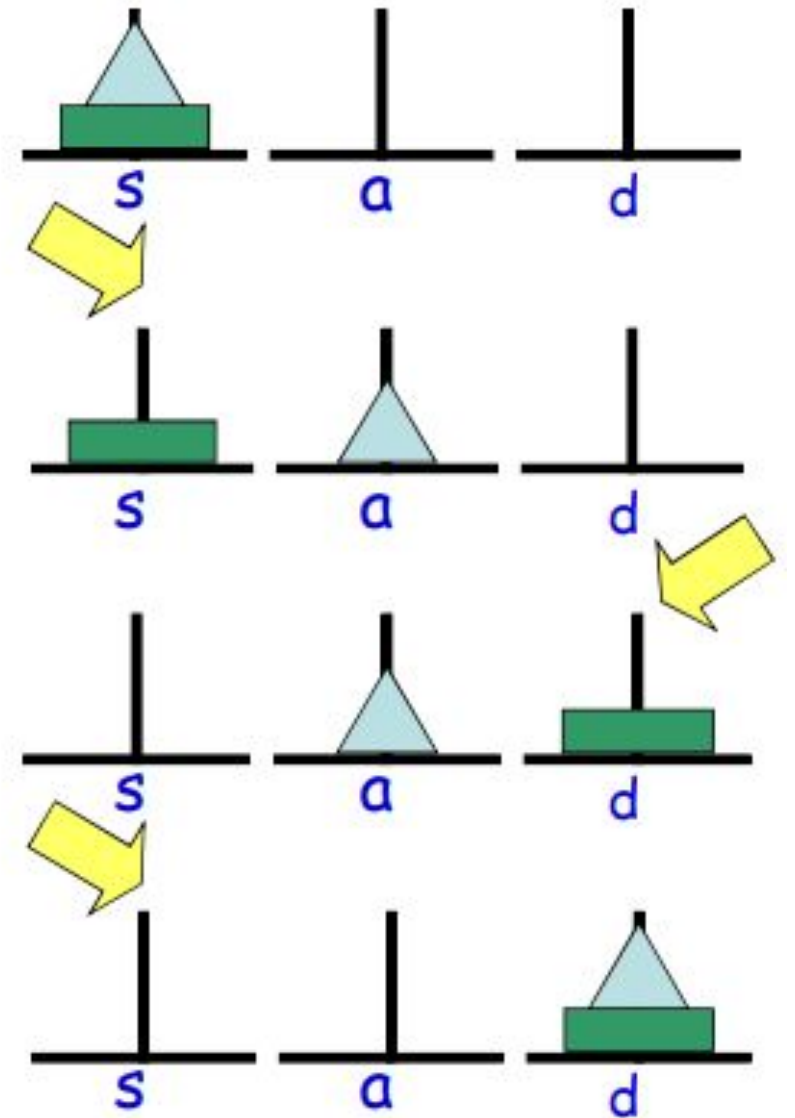


# Let's Make the Function Work

```
int Hanoi(int n, int s, int d, int a) {  
    if (n==1) {  
        print("%d -> %d", s, d);  
        return 1;  
    }  
    else {  
        x1=Hanoi(n-1, s, a, d);  
        print("%d -> %d", s, d); count++;  
        x2=Hanoi(n-1, a, d, s);  
        return x1+x2+1  
    }  
}
```

**BASE CASE:**

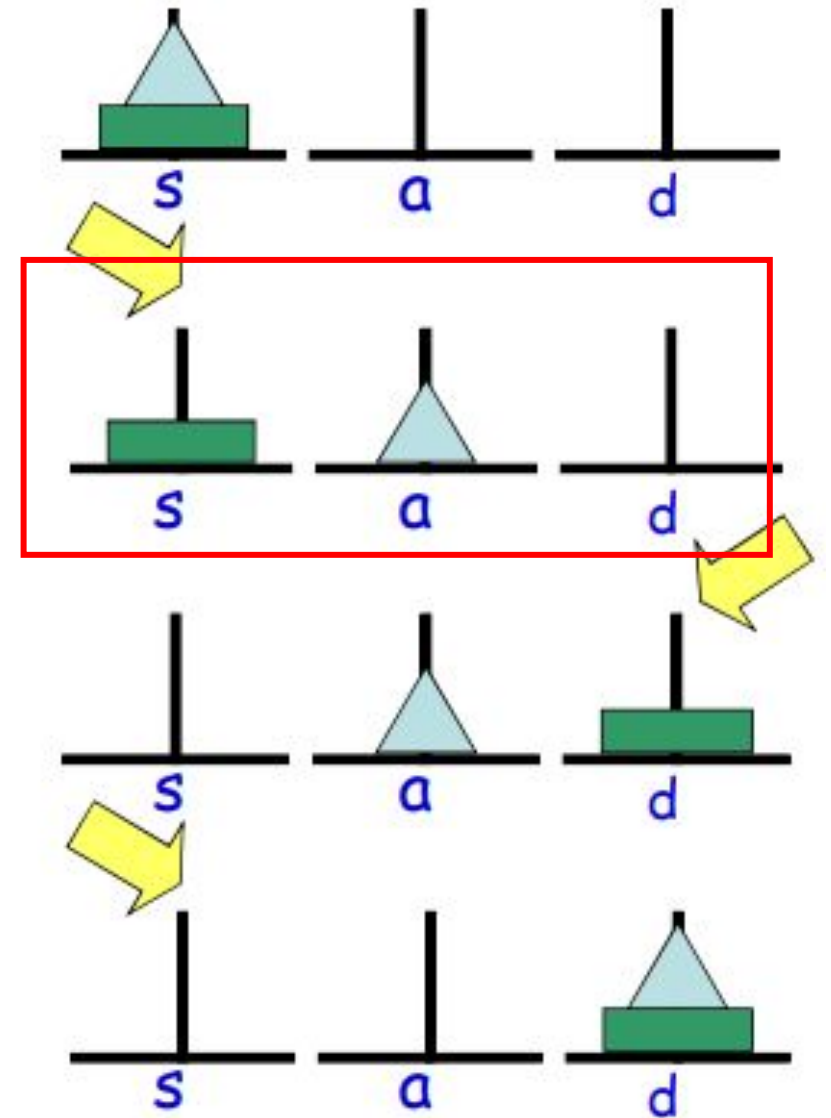
**Move 1 disc from s to d**



# Let's Make the Function Work

```
int Hanoi(int n, int s, int d, int a) {  
    if (n==1) {  
        print("%d -> %d", s, d);  
        return 1;  
    }  
    else {  
        x1=Hanoi(n-1, s, a, d);  
        print("%d -> %d", s, d); count++;  
        x2=Hanoi(n-1, a, d, s);  
        return x1+x2+1  
    }  
}
```

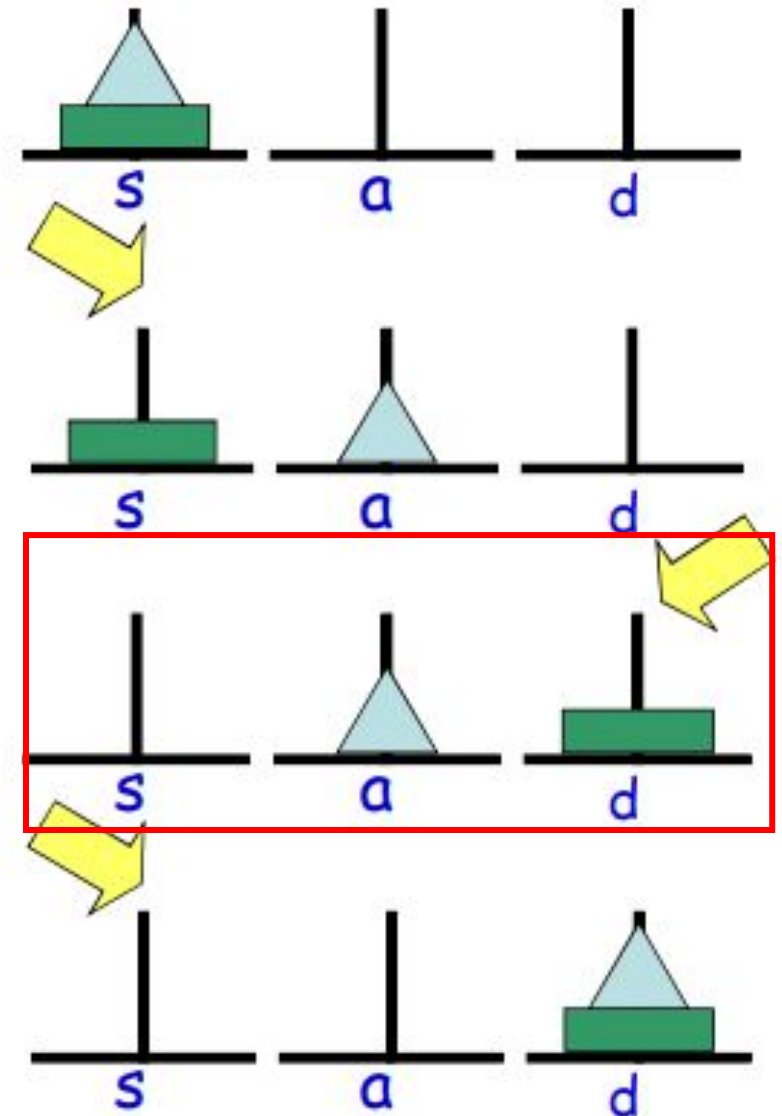
**Move n-1 disc from s to a with the help of d**



# Let's Make the Function Work

```
int Hanoi(int n, int s, int d, int a) {  
    if (n==1) {  
        print("%d -> %d", s, d);  
        return 1;  
    }  
    else {  
        x1=Hanoi(n-1, s, a, d);  
        print("%d -> %d", s, d); count++;  
        x2=Hanoi(n-1, a, d, s);  
        return x1+x2+1  
    }  
}
```

**Move n-1 disc from s to a with the help of d**  
**Move 1 disc from s to d (print and count)**



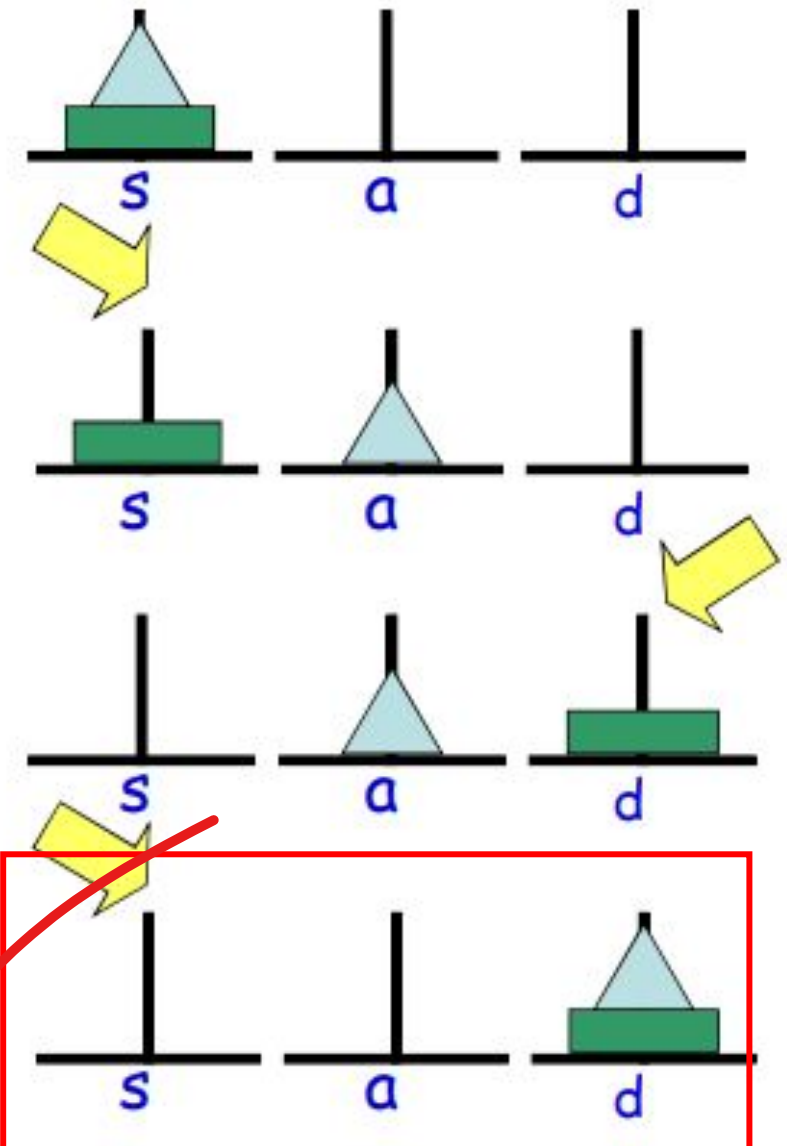
# Let's Make the Function Work

```
int Hanoi(int n, int s, int d, int a) {
    if (n==1) {
        print("%d -> %d", s, d);
        return 1;
    }
    else {
        x1=Hanoi(n-1, s, a, d);
        print("%d -> %d", s, d); count++;
        x2=Hanoi(n-1, a, d, s);
        return x1+x2+1;
    }
}
```

**Move n-1 disc from s to a with the help of d**

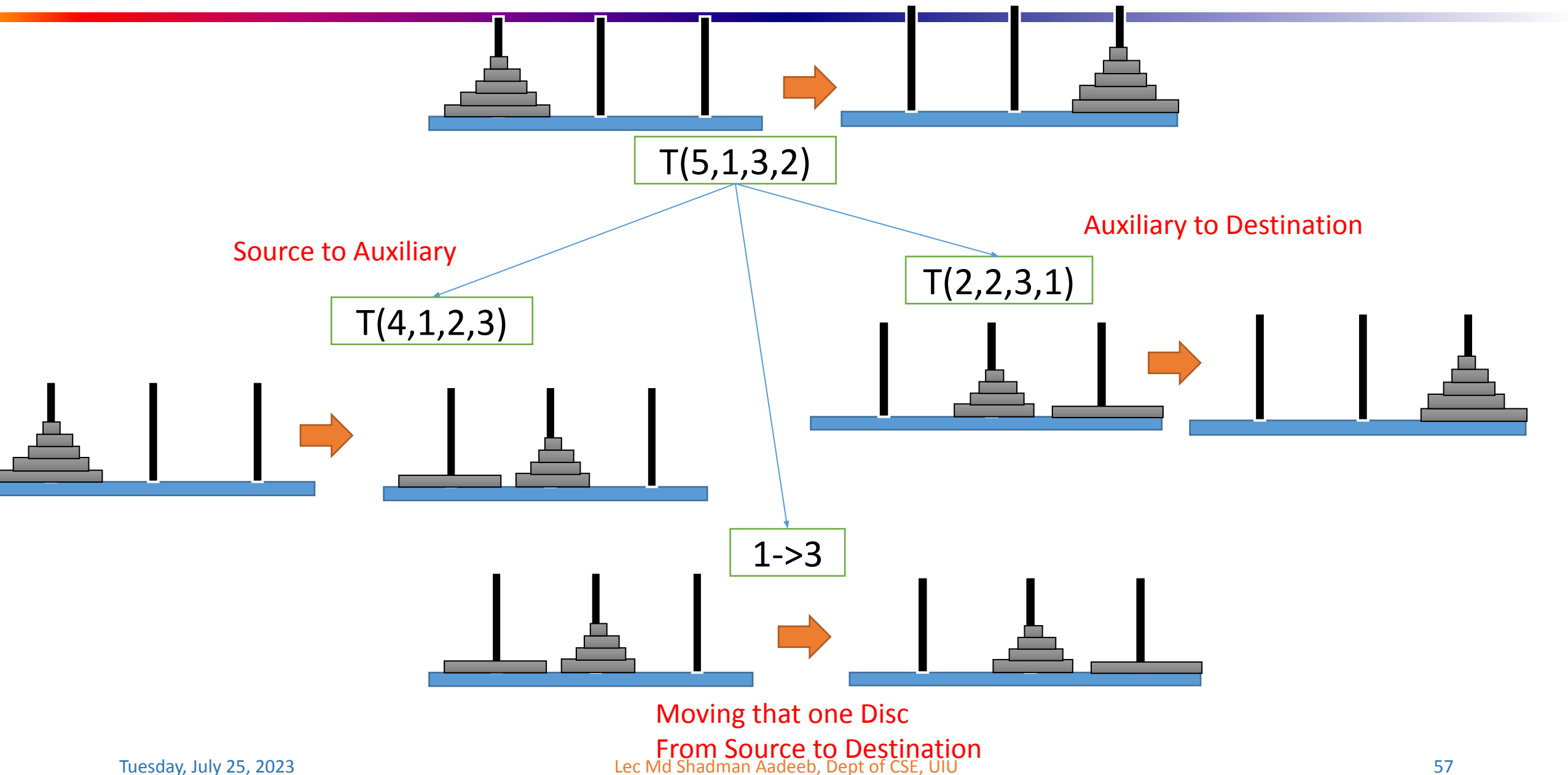
**Move 1 disc from s to d (print and count)**

**Move n-1 disc from a to d with the help of s**

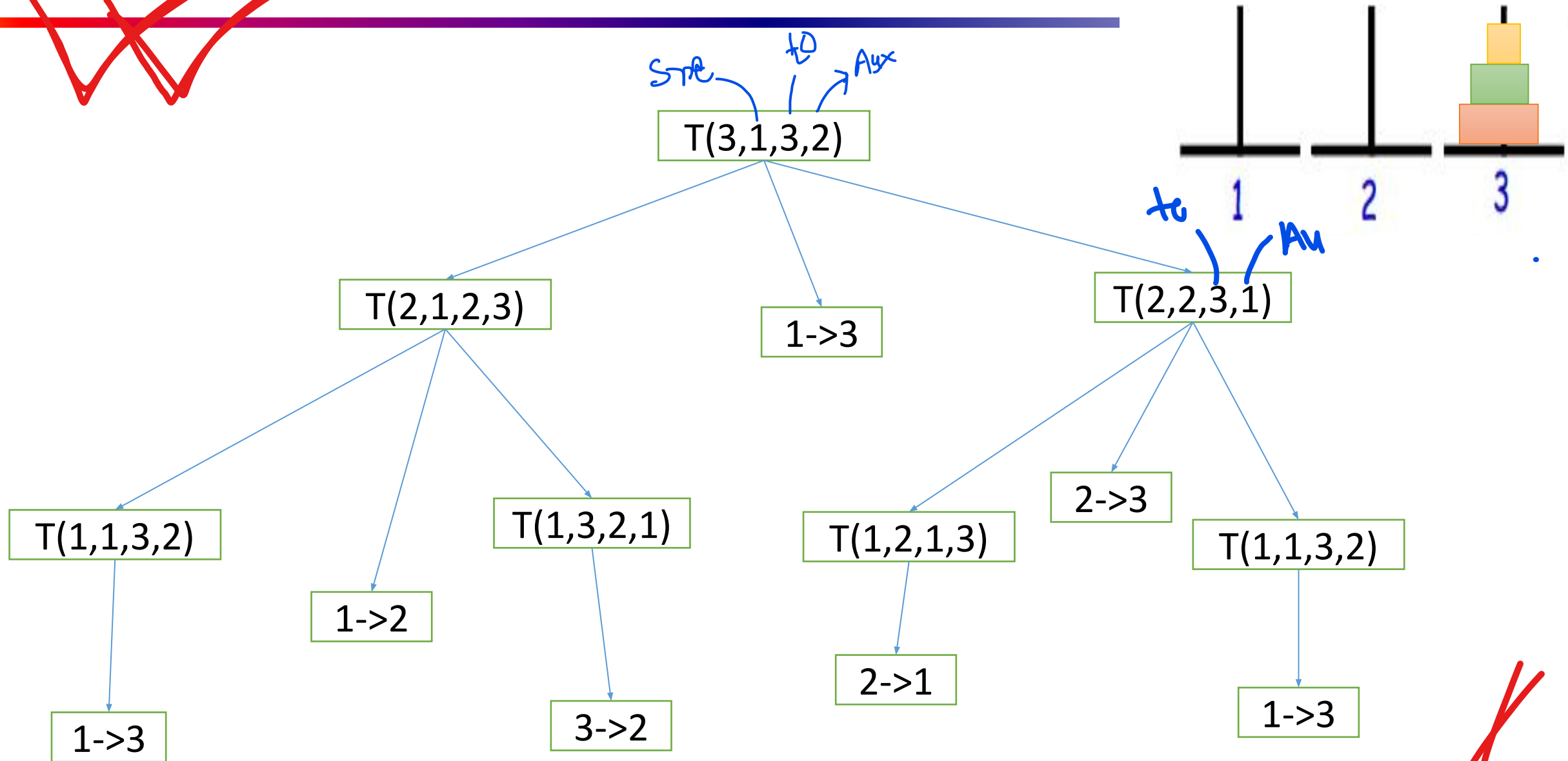




# Understanding how the recursion is working



# The Recursion Tree



# A formula for obtaining the total number of movements

No. of Disk Movements is  $a_k = 2a_{k-1} + 1$

$$a_1 = 1 = 2^1 - 1$$

$$a_2 = 2a_1 + 1 = 3 = 2^2 - 1$$

$$a_3 = 2a_2 + 1 = 2 \cdot 3 + 1 = 7 = 2^3 - 1$$

$$a_4 = 2a_3 + 1 = 2 \cdot 7 + 1 = 15 = 2^4 - 1$$

$$a_5 = 2a_4 + 1 = 2 \cdot 15 + 1 = 31 = 2^5 - 1$$

$$a_6 = 2a_5 + 1 = 2 \cdot 31 + 1 = 63 = 2^6 - 1$$

By guessing the pattern:  $a_k = 2^k - 1$

$a_k$

The number of  
Movements required to transfer  
K discs from one tower/peg  
To another

Can you prove this formula using induction?

---

# Thank You

